Yoobee College of Creative Innovation
Master of Software Engineering — Level 8

MSE802 Quantum Computing

Assessment 2 — Quantum Project (60%)

Designing, Running and Evaluating Quantum Circuits:

Bell States, Qiskit and QASM Circuits, Quantum Tic-Tac-Toe and a Quantum Classifier on the Quokka Device

GROUP PROJECT

Student 1	Anish Shrestha — Student ID 270889991
Student 2	Sandhya Tiwari — Student ID 270888992
Lecturer	Dr Helena Bahrami
Course	MSE802 Quantum Computing (15 credits)
Date	31 October 2026
GitHub repository	github.com/anishnz/quantumcomputing_YB_Anish
Referencing style	APA 7th edition

Auckland, New Zealand

Declaration of Originality

We, the undersigned members of this group, declare that this report and the accompanying code are our own group work, completed for MSE802 Assessment 2. All sources of information, code and ideas from others, including the lecture notes and the supplied lab notebooks, have been acknowledged and referenced in APA 7th style. This work has not been submitted for any other course. We understand Yoobee College's policies on plagiarism and collusion as described in the Student Handbook.

We confirm that the results reported here come from the notebooks submitted with this report. Results produced on a local simulator because the Quokka device was unavailable are clearly labelled as such and are not presented as device measurements.

Use of digital tools: Python libraries (Cirq, Qiskit, NumPy, Matplotlib) were used for all computation. A generative AI assistant was used to support code drafting, debugging and language editing; all code was run, checked and understood by the group, and the analysis and conclusions are our own. [Edit this sentence to match Yoobee's AI-use policy and your actual use.]

Name	Student ID	Signature / date
Anish Shrestha	270889991	
Sandhya Tiwari	270888992	

Table of Contents

Declaration of Originality.... 1
Table of Contents.... 2
Abstract.... 4
1. Introduction.... 5
1.1 What is this project about?.... 5
1.2 Which learning outcomes does each task show?.... 5
1.3 Which tools and device did we use?.... 5
1.4 How should the reader read the results?.... 6
2. Project Setup and How to Run.... 7
2.1 What is in the project folder?.... 7
2.2 How do we run it on a Windows PC?.... 7
2.3 Where can we see the output?.... 8
2.4 How do we run it in Google Colab?.... 8
2.5 Why could we not reach the Quokka device, and what does a login change?.... 9
2.6 What alternative code did we provide?.... 10
3. Task 1 – Entanglement Demonstration (Cirq + Quokka).... 11
3.1 What is a Bell state, and why is it special?.... 11
3.2 Which gates make the state, and why?.... 11
3.3 How did we build it in Cirq?.... 11
3.4 What did the measurement show?.... 12
3.5 Why is the split not exactly 500/500?.... 12
3.6 Would a real superconducting or ion-trap computer give the same result?.... 13
3.7 What does Task 1 prove?.... 13
4. Task 2 – Qiskit Circuits and a Custom QASM Circuit.... 14
4.1 Part A – What is block "C" in the given circuit?.... 14
4.2 What should the circuit do? (Step-by-step maths).... 14
4.3 What did Quokka (or the labelled simulator) return?.... 14
4.4 Part B – Why did we design a quantum half adder?.... 15
4.5 What does the theory predict?.... 16
4.6 Did the measurement agree?.... 16
4.7 Can Quokka run every gate in our design?.... 17
4.8 What did Task 2 teach us? (Critical evaluation).... 17
5. Task 3 – Investigating a Quantum Code: Quantum Tic-Tac-Toe.... 18
5.1 What is the purpose of the program?.... 18
5.2 How is the code organised?.... 18
5.3 What was missing, and what did we complete or fix?.... 18
5.4 How does a move become a gate? (Circuit generation).... 19
5.5 How does a rotation move change the odds?.... 19
5.6 What happened when we played? (Four recorded games).... 20
5.7 What do the games show? (Analysis).... 21
5.8 Did we also play the game by clicking the buttons?.... 22
5.9 Can the game circuit run on Quokka?.... 23
5.10 Which quantum gates does the game use? (Gate section).... 23
5.11 How do these gates fit the wider theory of quantum logic gates?.... 24
5.12 What are the strengths and limits of the program?.... 24
6. Task 4 – Machine Learning Quantum Analysis.... 25
6.1 What problem does the code solve? (Problem formulation).... 25
6.2 Which part of the code is the input to the circuit?.... 25
6.3 How does the circuit work? (Quantum part, excluding the optimiser).... 25
6.4 What did we add to monitor every iteration?.... 26

6.5 What do the two required graphs show?.....27
6.6 How did we make the code run without the quantum computer or circuit?.....28
6.7 What were the differences in efficiency and effectiveness?.....28
6.8 Did anything not work as expected?.....30
6.9 What is our overall judgement for Task 4?.....30
7. Critical Discussion Across the Four Tasks.....31
7.1 How does the evidence meet each learning outcome?.....31
7.2 What are the main limitations of our work?.....31
7.3 What would we do next?.....31
8. Development Process and Code Walkthrough.....32
8.1 What process did we follow?.....32
8.2 How does the shared helper work?.....32
8.3 What does each notebook cell do?.....33
8.4 What went wrong during development, and how did we fix it?.....34
8.5 How did the code change over time? (Version history).....35
9. Conclusion.....36
References.....37
Appendix A – Submission Structure and How to Run.....39
Appendix B – Key Code Excerpts.....39

(If page numbers are missing, right-click the table and choose Update Field.)

Abstract

This group project designs, runs and evaluates quantum circuits for the four tasks of MSE802 Assessment 2. Every circuit was written in Python (Cirq or Qiskit) or directly in OpenQASM 2.0 and executed with **1000 shots**. In **Task 1**, a Hadamard gate and a CNOT gate prepare the Bell state (|00⟩ + |11⟩)/√2; all 1000 shots gave correlated results (508 × 00, 492 × 11). In **Task 2**, the given H–X–C circuit was identified as an H gate and an X gate feeding a CNOT. It produces the second Bell state (|01⟩ + |10⟩)/√2, and our own four-qubit QASM circuit, a **quantum half adder**, computed all four one-bit additions in a single circuit with 1000 correct results out of 1000. In **Task 3**, we completed the quantum tic-tac-toe game (four missing gates, the eight winning lines and four bug fixes), analysed four recorded games and played three more by clicking the game's buttons in Jupyter. The analysis shows how each click becomes a gate and why a NOT gate has no effect on an undecided tile. In **Task 4**, we analysed a tree-shaped variational classifier, logged the output and time of all 100 training iterations, and compared it with three local versions. The circuit reached zero training loss and classified all four images. Removing the quantum device made each iteration about **470 times faster**, while linear classical models failed because the data is an XOR-type problem. The Quokka endpoint was offline when the results in this draft were produced (ERR_NGROK_3200), so these results come from a local simulator and are clearly labelled; one script re-runs everything on Quokka. Sources are cited in APA 7th style.

Keywords: Bell state, entanglement, Cirq, Qiskit, OpenQASM, Quokka, half adder, quantum tic-tac-toe, variational quantum classifier, SPSA

1. Introduction

1.1 What is this project about?

The aim of Assessment 2 is to **implement different quantum circuits, test programs and algorithms on a practical quantum computer, and judge how effective these systems are**. The four tasks move from a two-qubit circuit (Task 1) to a hand-written circuit (Task 2), to reading someone else's quantum code (Task 3), and finally to a small quantum machine-learning model (Task 4). Each task is answered with working code, real measurements, an expected-versus-measured comparison and a short critical discussion.

1.2 Which learning outcomes does each task show?

Table 1. *Assessment tasks, weights and learning outcomes*

Task	Topic	Weight	LOs	Main evidence in this report
Task 1	Entanglement demonstration (Cirq)	25%	LO2	Bell circuit, state maths, 1000-shot results, statistics
Task 2	Qiskit circuit + own QASM circuit	35%	LO1, LO2	Gate-by-gate maths, two circuits, theory vs measurement
Task 3	Investigate quantum tic-tac-toe	25%	LO1, LO2, LO4	Completed code, four recorded games, gate section
Task 4	Machine-learning quantum analysis	15%	LO1, LO2, LO4	Circuit analysis, iteration log, two graphs, local comparison

1.3 Which tools and device did we use?

All code runs in Jupyter notebooks (VS Code or Jupyter) with Python 3.12 or newer. Circuits are built with **Cirq** 1.7 (Task 1; Cirq Developers, 2026) and **Qiskit** 2.6 (Tasks 2–4; Javadi-Abhari et al., 2024) and sent to the device as **OpenQASM 2.0** text (Cross et al., 2017). The device is **Quokka**, a personal quantum computer developed at the University of Technology Sydney. Quokka is an **emulator**: classical hardware that reproduces the behaviour of an ideal, fault-tolerant 30-qubit quantum computer (University of Technology Sydney [UTS], 2024). The course notebook and the official Quokka notebooks reach it with an HTTP request containing a QASM script and a shot count, `POST /qsim/qasm` with `{"script": ..., "count": 1000}` (Bahrami, 2026d; Quokka Computing, n.d.-a). The Quokka docs list the gates it accepts (h, x, y, z, rx, ry, rz, cx, swap, cphase, U), so our helper also rewrites any other gate into this set before sending (Quokka Computing, n.d.-b). This matters for the analysis: because Quokka emulates an *error-free* machine, its results should match ideal theory except for random shot noise, unlike today's noisy hardware (Preskill, 2018).

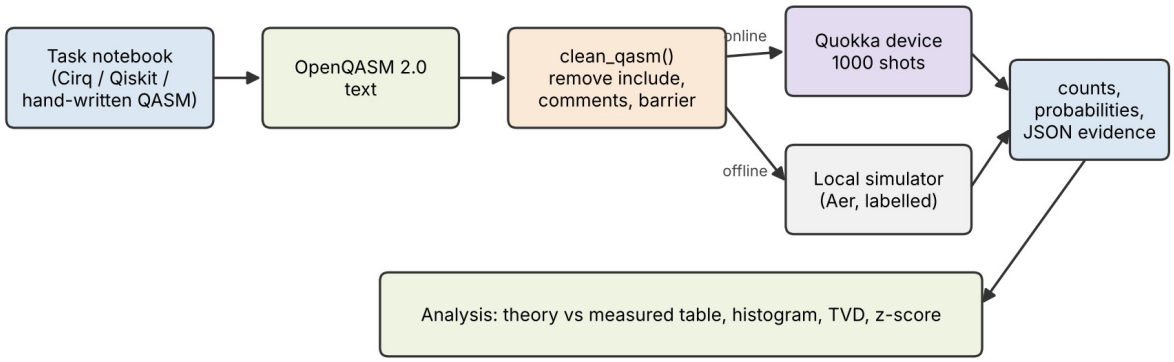


Figure 1. *Shared execution pipeline used by Tasks 1, 2 and 4 (file common/quokka_helper.py)*

To avoid repeating code, we wrote one shared helper (Figure 1). It (1) cleans the QASM text, removing the `include "qelib1.inc"` line, comments and barriers that Quokka does not accept; (2) posts it to Quokka with 1000 shots; (3) turns the raw shot list into counts; and (4) saves every run as JSON evidence with its source, time stamp and run time.

> Honesty note on the Quokka device. On 30–31 October 2026 the Quokka address replied: "endpoint quokka1.quokkacomputing.com is offline (ERR_NGROK_3200)". Following the course guidance that simulator results are acceptable, the helper therefore ran the identical, Quokka-compatible QASM scripts on the Qiskit Aer simulator with a fixed seed, and every result in this draft is labelled LOCAL SIMULATOR in the notebooks and JSON files. Because Quokka itself emulates an ideal machine, the expected behaviour is the same, but these numbers are not Quokka data. Running run_all.bat when Quokka is online replaces them with device results and the label changes to QUOKKA DEVICE.

1.4 How should the reader read the results?

Bit-strings follow the Qiskit convention: the **left-most character is the highest-numbered bit**. Two measures are used to compare theory with measurement. The **total variation distance (TVD)** is half the sum of the absolute differences between two probability distributions: 0 means identical and 1 means completely different. For a 50/50 outcome, the **z-score** tells us whether a count is within normal shot noise: with 1000 shots the standard deviation is $\sqrt{(1000 \times 0.5 \times 0.5)} \approx 15.8$, so counts between about 468 and 532 are normal.

2. Project Setup and How to Run

All code, results and this report are stored in the GitHub repository **https://github.com/anishnz/quantumcomputing_YB_Anish** in the folder `Assignment_2_Quantum`. The project can be run in four ways: one click on Windows, one command on any computer, cell by cell in VS Code or Jupyter, or fully online in **Google Colab** with no installation. Every route runs the same notebooks with **1000 shots** per circuit and saves the same evidence files.

2.1 What is in the project folder?

Table 2. *Project folders and files (one folder per task, as the brief requires)*

Folder / file	What it contains
Task_1_Bell_State/	Task 1 notebook (Cirq Bell state) and results/ (counts JSON, table CSV, histogram, QASM)
Task_2_Qiskit_QASM/	Task 2 notebook, our custom_half_adder.qasm, results/ (circuits, counts, histograms)
Task_3_Quantum_TicTacToe/	completed game notebook, the original supplied notebook, results/ (game circuits, summaries)
Task_4_Quantum_ML/	modified ML notebook (.ipynb, as required), the original notebook, results/ (iteration logs, graphs)
Alternative_Code/	a second way to solve each task: Qiskit Bell state, Cirq circuits, terminal game, Statevector classifier
common/quokka_helper.py	shared code: clean QASM, send to Quokka, labelled fallback, counts, charts
run_project.py / run_project.bat / run_all.bat	the runner: menu, all tasks together, one task, or the alternative code
Colab_Run_All_Tasks.ipynb	runs the whole project in Google Colab
requirements.txt, MAINTENANCE.md, README.md	package versions, maintenance guide, step-by-step instructions
Report/	this report (Assignment2.docx and Assignment2.pdf) and its figures

2.2 How do we run it on a Windows PC?

1. Install **Python 3.12 or newer** (tick "Add to PATH") and download the repository: green **Code** button → **Download ZIP**, or `git clone https://github.com/anishnz/quantumcomputing_YB_Anish.git`.
2. Open the `Assignment_2_Quantum` folder and **double-click `run_project.bat`**. The first time, it creates a virtual environment and installs `requirements.txt` (a few minutes).
3. A menu appears: **1–4** runs one task, **A** runs Tasks 1–4 together, **L** runs the alternative code, **S** tests the Quokka connection. `run_all.bat` runs all four tasks without the menu.
4. From a terminal the same runner takes arguments: `python run_project.py all`, `python run_project.py 2` or `python run_project.py alt`. On macOS/Linux use `./run_project.sh all`.

```
C:\...\Assignment_2_Quantum> run_project.bat all
Quokka device: offline (Quokka endpoint is offline (HTTP 404, ERR_NGROK_3200))
  -> circuits will run on the local simulator and be labelled LOCAL SIMULATOR
======================================================================
RUNNING Task 1 - Bell state (Cirq)
  finished in 3.9 s
  output saved in the notebook and as HTML: outputs_html/Task_1_Bell_State.html
  RESULTS:
    counts (1000 shots): {'00': 508, '11': 492}   expected: only 00 and 11
RUNNING Task 2 - Qiskit circuit + custom QASM
    2A H-X-CNOT counts: {'01': 492, '10': 508}   expected: only 01 and 10
    2B half adder counts: {'0000': 250, '0111': 244, '1001': 247, '1010': 259}
RUNNING Task 3 - Quantum tic-tac-toe
    game2_diagonals  X wins  705/1000   O wins  210/1000
    game3_not_gate   X wins  135/1000   O wins  694/1000
RUNNING Task 4 - Quantum machine learning
    Quantum circuit (local sim, 1000 shots)   0.56269 s/iter   final loss 0.0
    Local-A: same circuit, exact NumPy        0.00108 s/iter   final loss 0.0003
======================================================================
ALL FOUR TASKS FINISHED. Open outputs_html/ to see every result.
```

Figure 2. *Terminal output of run_project.bat all (shortened): Quokka status, each task's run and its key results*

2.3 Where can we see the output?

- **In the terminal:** after each task, the runner prints the key numbers (Figure 2).
- **Inside each notebook:** circuits, tables, histograms and graphs are saved in the executed `.ipynb`, ready to open in VS Code, Jupyter or Colab.
- **As web pages:** `outputs_html/` contains one HTML file per task; double-click to open in any browser.
- **As evidence files:** each `results/` folder stores the raw counts with their source (QUOKKA DEVICE or LOCAL SIMULATOR), CSV tables and PNG charts used in this report.

2.4 How do we run it in Google Colab?

Colab runs the notebooks on Google's computers, so nothing needs to be installed. The repository is connected to Colab through its GitHub address:

1. Open **https://colab.research.google.com/github/anishnz/quantumcomputing_YB_Anish/blob/main/Assignment_2_Quantum/Colab_Run_All_Tasks.ipynb** (or, in Colab, choose **File → Open notebook → GitHub**, enter `anishnz/quantumcomputing_YB_Anish` and pick `Colab_Run_All_Tasks.ipynb`).
2. Click **Runtime → Run all**. The notebook clones the repository, installs Cirq and Qiskit, tests Quokka, runs Tasks 1–4 together, shows the results and charts, runs the alternative code, and offers a zip of all outputs.
3. To run one task, open its notebook from GitHub the same way. Its first code cell detects Colab, downloads the project and moves into the task folder, so the rest runs unchanged.

Colab test run (31 October 2026). The notebook was saved to Google Drive as a Colab project named `Assignment2_MSE802_Quantum_Project.ipynb` and run with **Runtime → Run all**. All four tasks finished without errors (Figure 3). Tasks 1–3 gave exactly the same counts as on the PC, because the local simulator uses a fixed seed. Task 4 took 160 s instead of about 90 s (0.99 s instead of about 0.6 s per iteration), because Colab's free CPU is slower than the PC; the loss curve and accuracy were identical. A first attempt showed one portability problem: the notebooks were executed by a Jupyter kernel without Cirq installed. We fixed it by making `run_project.py` install any missing packages into its own Python and register a matching kernel.

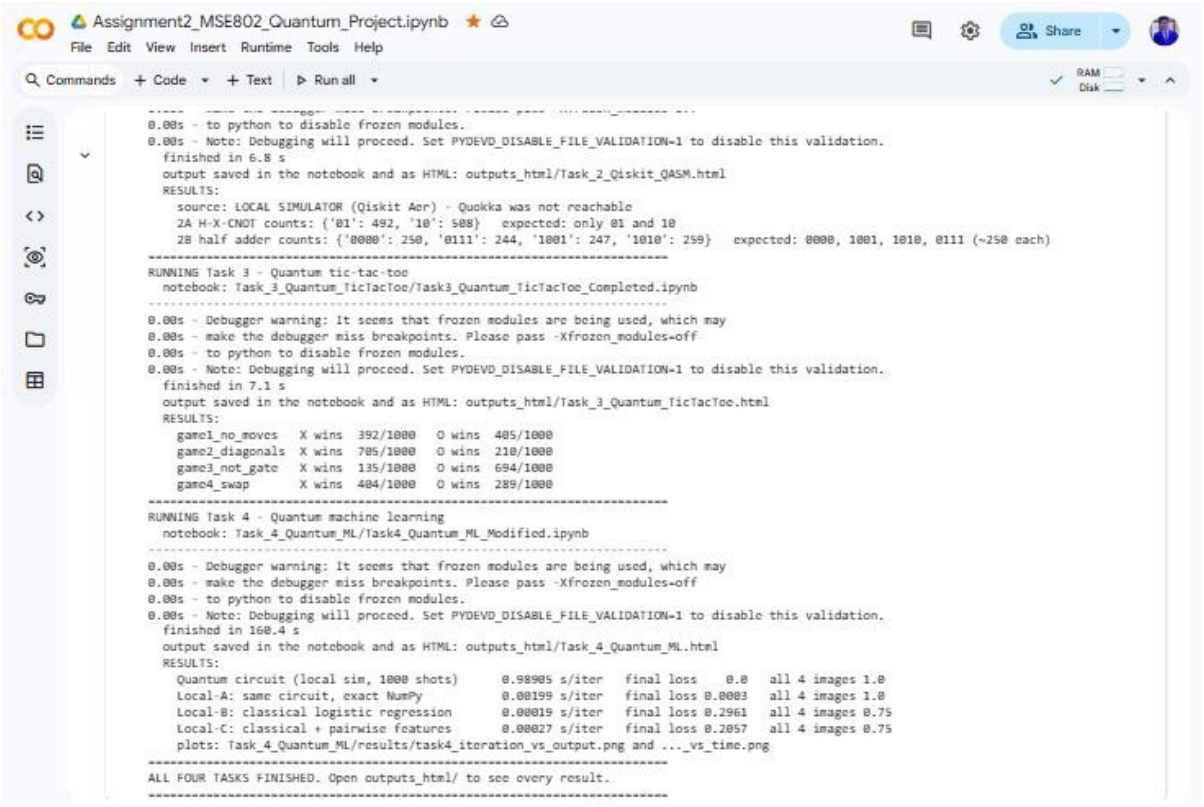


Figure 3. *Tasks 1–4 run together in Google Colab (notebook Assignment2_MSE802_Quantum_Project.ipynb, Quokka offline so labelled LOCAL SIMULATOR)*

2.5 Why could we not reach the Quokka device, and what does a login change?

Quokka has two parts that are easy to confuse. The **Quokka notebooks** (Start Here, Docs, the lab notebooks) are public Google Colab pages, and anyone can open them. They are only instructions. The **Quokka simulator** is a separate server, reached at `http://quokka1.quokkacomputing.com/qsim/qasm` through a tunnelling service called ngrok (Quokka Computing, n.d.-a, n.d.-b). The server must be switched on by its owner. From 30 October 2026 every request returned **HTTP 404 with the code ERR_NGROK_3200**, which ngrok uses to mean that the address still exists but no server is connected behind it.

Table 3. *Quokka connection tests (30–31 October 2026)*

Test	Where it ran	Result
POST /qsim/qasm (2-qubit Bell script)	project workspace (Python requests)	HTTP 404, ERR_NGROK_3200
Same, through run_project.py status	Google Colab (Google's network)	HTTP 404, ERR_NGROK_3200
Open the address in a web browser	student PC, Claude desktop browser	"The endpoint quokka1.quokkacomputing.com is offline"
quokka2, quokka3, quokka4, quokka (http and https)	project workspace	all ERR_NGROK_3200
Automatic re-check every 2 minutes	background watcher on 31 October	see results/ JSON source labels

Three conclusions follow. (1) The problem is not our code: the same request format, `{"script": ..., "count": ...}`, is used in the official Quokka notebooks, and the identical error was seen from three

different networks. (2) **A login or API key would not help.** The official documentation sends no key or token with any request; the only setting is the name of the Quokka ("default Quokka address (change if you have your own!)"). (3) If the class is given its own Quokka, the only change needed is its name in `quokka_config.json` (or the `QUOKKA_NAME` environment variable). Following the course guidance, the results in this report come from the labelled local simulator, which runs exactly the same Quokka-compatible QASM.

2.6 What alternative code did we provide?

Table 4. *Alternative implementations in Alternative_Code/ (run with python run_project.py alt)*

Task	Alternative	Why it is useful
Task 1	Bell state written in Qiskit instead of Cirq	shows the result does not depend on the library; same QASM reaches Quokka
Task 2	Both circuits written in Cirq and run on Cirq's own simulator	independent check of Part A and the half adder
Task 3	Text-only tic-tac-toe in the terminal (X n, O n, NOT n, SWAP a b, PROB, MEASURE)	plays without widgets; --demo replays a game
Task 4	Same classifier as a Qiskit circuit, exact Statevector output, COBYLA optimiser	shows the circuit, not the optimiser, solves the problem (loss 0 in 87 evaluations)

3. Task 1 – Entanglement Demonstration (Cirq + Quokka)

> Task question: Create an entangled Bell state using Cirq and run the system on the Quokka device. The Bell state is $|\psi\rangle = (1/\sqrt{2})(|00\rangle + |11\rangle)$. Generate qubits and apply the necessary gates to generate the state, and generate the necessary results. Notebook: Task_1_Bell_State/Task1_Bell_State_Cirq_Quokka.ipynb

3.1 What is a Bell state, and why is it special?

A **qubit** is a two-level quantum system whose state is $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, where α and β are complex amplitudes with $|\alpha|^2 + |\beta|^2 = 1$ (Nielsen & Chuang, 2010; Yanofsky & Mannucci, 2008). Two qubits live in a four-dimensional space with basis $|00\rangle$, $|01\rangle$, $|10\rangle$ and $|11\rangle$. The Bell state asked for in the task is

$$|\Phi^+\rangle = (1/\sqrt{2})\ (\ |00\rangle + |11\rangle\ )$$

It is **entangled**: it cannot be written as (state of qubit 0) $\otimes$ (state of qubit 1). Suppose it could; then $(a|0\rangle + b|1\rangle) \otimes (c|0\rangle + d|1\rangle)$ would need $ac = bd = 1/\sqrt{2}$ but $ad = bc = 0$, which is impossible. In practice, each qubit on its own is a fair coin, but the two coins always land the same way. Einstein, Podolsky and Rosen (1935) used such correlations to question quantum theory; Bell (1964) showed they cannot be explained by hidden classical values, and loophole-free experiments have confirmed this (Hensen et al., 2015).

3.2 Which gates make the state, and why?

Two gates are enough, as taught in the course (Bahrami, 2026a). The **Hadamard (H)** gate creates an equal superposition, and the **controlled-NOT (CNOT)** gate copies the control's value into the target *for every branch of the superposition at once*, which links (entangles) the qubits.

$$H = (1/\sqrt{2})\ [\ [1,\ 1],\ [1,\ -1]\ ] \qquad \mathrm{CNOT} = [\ [1,0,0,0],\ [0,1,0,0],\ [0,0,0,1],\ [0,0,1,0]\ ]$$

Table 5. *State of the two qubits after each gate (written $|q0\ q1\rangle$)*

Step	Gate	State
0	start	$\|00\rangle$
1	H on q0	$(1/\sqrt{2})(\|0\rangle + \|1\rangle) \otimes \|0\rangle = (1/\sqrt{2})(\|00\rangle + \|10\rangle)$
2	CNOT (control q0, target q1)	$(1/\sqrt{2})(\|00\rangle + \|11\rangle) = \|\Phi^+\rangle$
3	measure both	00 with probability $\|1/\sqrt{2}\|^2 = 0.5$; 11 with probability 0.5

3.3 How did we build it in Cirq?

The Cirq code creates two `LineQubit` objects, applies `cirq.H(q0)` and `cirq.CNOT(q0, q1)`, and measures both qubits into one register. Before using the device, the notebook simulates the circuit without measurement to obtain the exact amplitudes: 0.7071 for $|00\rangle$ and $|11\rangle$ and 0 for the others, matching Table 5. The circuit is then exported with `cirq.qasm()`, cleaned by the helper and sent with 1000 shots.

```
q0, q1 = cirq.LineQubit.range(2)          # two qubits, both start in |0>
bell = cirq.Circuit(
    cirq.H(q0),                           # superposition on q0
    cirq.CNOT(q0, q1),                    # entangle q1 with q0
    cirq.measure(q0, q1, key="c"))       # read both qubits
run = qh.run_qasm(qh.clean_qasm(cirq.qasm(bell)), shots=1000)
```

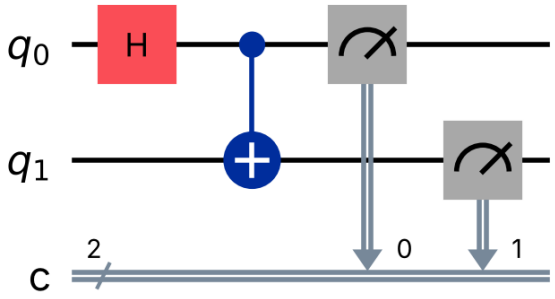


Figure 4. *Task 1 Bell-state circuit: H on q0, CNOT from q0 to q1, then measurement*

3.4 What did the measurement show?

Table 6. *Task 1 results: theory versus 1000-shot measurement (source: local simulator, Quokka offline)*

Outcome	Theory P	Counts (of 1000)	Measured P	Difference
00	0.500	508	0.508	+0.008
01	0.000	0	0.000	0.000
10	0.000	0	0.000	0.000
11	0.500	492	0.492	−0.008

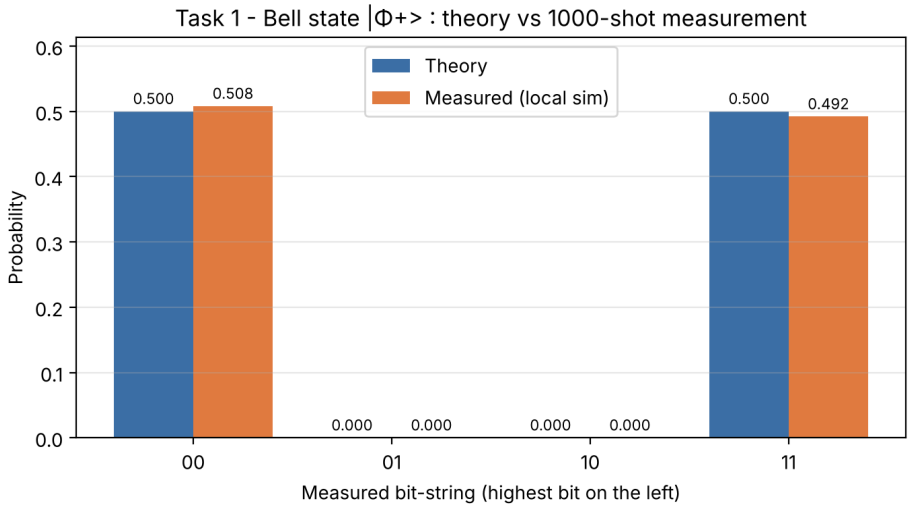


Figure 5. *Task 1 histogram: theory and measured probabilities for each outcome*

All 1000 shots were **correlated** (00 or 11), and none were 01 or 10. The 00 count of 508 is only 0.51 standard deviations above 500 (z = +0.51), and the TVD between theory and measurement is 0.008. The result therefore agrees with the Bell state within normal shot noise.

3.5 Why is the split not exactly 500/500?

Measurement is random. Each shot is an independent trial with p = 0.5, so the 00 count follows a binomial distribution with mean 500 and standard deviation 15.8. A perfect 500/500 split would itself be unusual (it happens only about 2.5% of the time). Small differences are **expected** and are not errors.

3.6 Would a real superconducting or ion-trap computer give the same result?

Not exactly. Physical devices have gate errors, decoherence and readout errors, which produce a few percent of 01 and 10 outcomes (Preskill, 2018). Quokka is designed to emulate an ideal fault-tolerant machine (UTS, 2024), so Quokka results should look like Table 6, with zero 01/10 counts. If 01 or 10 appear when the notebook is re-run on Quokka, the first things to check are the qubit order in the QASM and the bit order in the reply. The helper checks bit order by reading each shot as [c0, c1] and reversing it to the Qiskit convention.

3.7 What does Task 1 prove?

- The circuit H → CNOT → measure correctly prepares $|\Phi^+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$ (LO2).
- Each qubit alone looks random (50/50), but the pair is perfectly correlated, which is the measurable signature of entanglement.
- Agreement between theory and measurement is quantified (TVD = 0.008, z = +0.51), not just claimed.

4. Task 2 – Qiskit Circuits and a Custom QASM Circuit

> Task question: Using Qiskit, generate the given circuit (H on one line, X on the other, both entering block C). Measure the output by running it on a Quokka device and print the output. Then write a QASM script for a circuit of your own using different qubits and various gates, run it on Quokka and print the output.
> Notebook: Task_2_Qiskit_QASM/Task2_Qiskit_Circuits_and_QASM.ipynb

4.1 Part A – What is block "C" in the given circuit?

The brief draws an H gate on the top line and an X gate on the bottom line, both entering a box labelled **C** that has **two inputs and one output arrow** (on the lower line). This is the standard way of drawing a **controlled-NOT** used as a function block: the top line is the *control*, the bottom line is the *target*, and the target carries the result (Nielsen & Chuang, 2010). We therefore implemented C as `cx(0, 1)`, and the lecturer confirmed that block C is a CNOT. Because the diagram shows a single output, we ran two versions: one measuring both qubits (full analysis) and one measuring only the output line (exactly as drawn).

4.2 What should the circuit do? (Step-by-step maths)

Table 7. *Task 2A state after each gate (Qiskit order |q1 q0⟩)*

Step	Gate	State |q1 q0⟩
0	start	|00⟩
1	H on q0	(1/√2)(|00⟩ + |01⟩)
2	X on q1	(1/√2)(|10⟩ + |11⟩)
3	C = CNOT (q0 → q1)	(1/√2)(|10⟩ + |01⟩) = |Ψ⁺⟩

The final state is the Bell state **|Ψ⁺⟩**: the two qubits are entangled and **always opposite**. The X gate is what turns the Task 1 state (always equal) into this one (always different). The output line q1 equals NOT(q0), so it is a fair 50/50 random bit. Predictions: 01 and 10 at 0.5 each, 00 and 11 never; the output line alone gives 0 and 1 at 0.5 each.

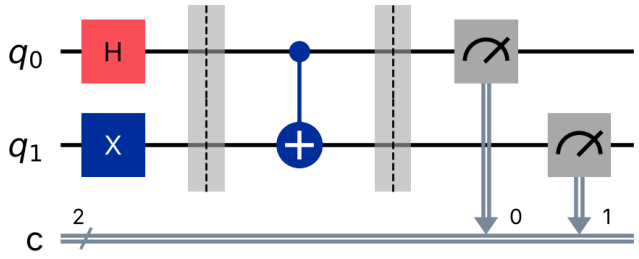


Figure 6. *Task 2A circuit in Qiskit: H and X, then block C implemented as a CNOT*

4.3 What did Quokka (or the labelled simulator) return?

Table 8. *Task 2A results, both qubits measured (1000 shots)*

Outcome |q1 q0⟩	Theory P	Counts (of 1000)	Measured P	Difference
00	0.000	0	0.000	0.000
01	0.500	492	0.492	−0.008

Outcome |q1 q0⟩	Theory P	Counts (of 1000)	Measured P	Difference
10	0.500	508	0.508	+0.008
11	0.000	0	0.000	0.000

The output-only version returned **0: 491 and 1: 509**. Both runs agree with theory: only anti-correlated outcomes appear, and the TVD is 0.008. The printed output in the notebook is the counts dictionary, `{'01': 492, '10': 508}`.

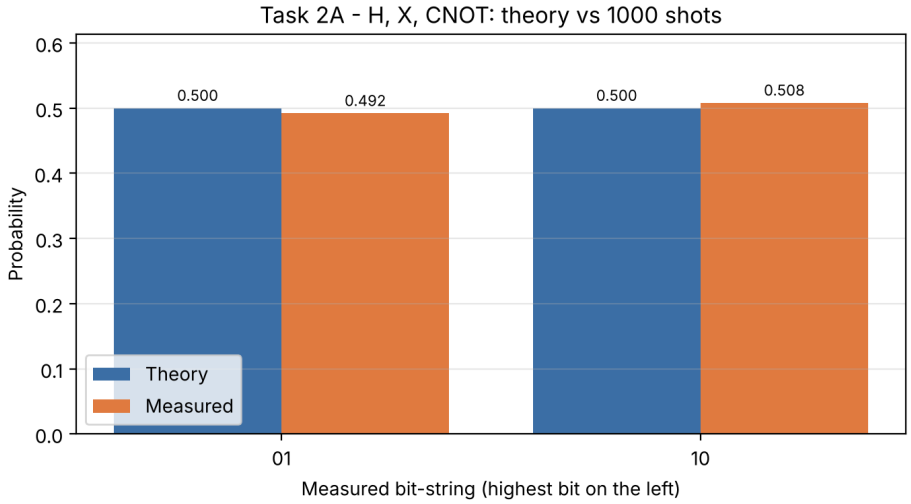


Figure 7. *Task 2A histogram: theory versus 1000-shot measurement*

4.4 Part B – Why did we design a quantum half adder?

The brief asks for a circuit of our own with **different qubits and various gates**, and the rubric rewards creativity *and* understanding. We chose a circuit with a clear purpose that we could check fully: a **half adder**, the basic building block of computer arithmetic, which returns SUM = A XOR B and CARRY = A AND B. Quantum adders built from CNOT and Toffoli gates are well known (Vedral et al., 1996). Our twist is to place the inputs in **superposition**, so one circuit holds all four additions (0+0, 0+1, 1+0, 1+1) at the same time. Each shot then reveals one complete, correct row of the truth table.

Table 9. *Qubits and gates in our custom circuit (file custom_half_adder.qasm)*

Qubit	Role	Gates	Why this gate
q0	input A	H, Z	H makes A = 0 and 1 at once; Z adds a −1 phase to show that phase is invisible to direct measurement
q1	input B	H, S	H for superposition; S adds an i phase (same lesson)
q2	CARRY	CCX (Toffoli)	Flips to 1 only if A = 1 AND B = 1
q3	SUM	CX, CX	Two CNOTs give SUM = A ⊕ B

```
OPENQASM 2.0;
include "qelib1.inc";
qreg q[4];  creg c[4];
h q[0];  h q[1];          // A and B in superposition
z q[0];  s q[1];          // phase gates: change amplitudes, not probabilities
ccx q[0],q[1],q[2];       // CARRY = A AND B
```

```
cx q[0],q[3];  cx q[1],q[3];   // SUM = A XOR B
measure q[0] -> c[0]; measure q[1] -> c[1];
measure q[2] -> c[2]; measure q[3] -> c[3];
```

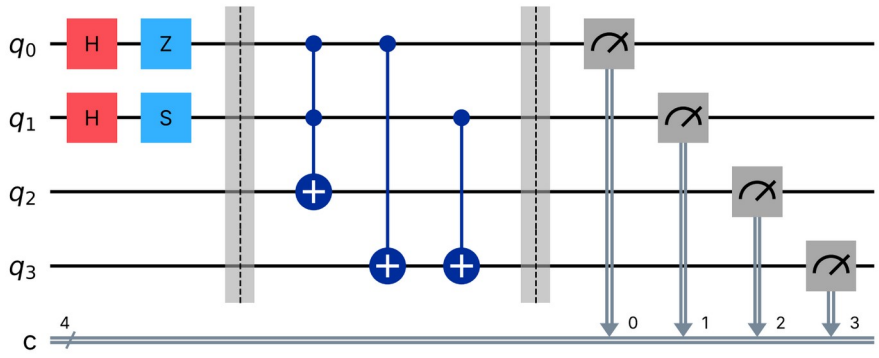


Figure 8. *Task 2B custom circuit (quantum half adder) loaded from our QASM file*

4.5 What does the theory predict?

After the two H gates the inputs are (1/2)(|00⟩ + |01⟩ + |10⟩ + |11⟩). The Z and S gates multiply some amplitudes by −1 or i. The state vector in the notebook shows amplitudes 0.5, −0.5i, −0.5 and 0.5i, but every probability is still $|\pm 0.5|^2 = |\pm 0.5i|^2 = 0.25$. The Toffoli and CNOT gates then write CARRY and SUM. Writing strings as c3c2c1c0 = SUM, CARRY, B, A, only four strings are possible: **0000, 1001, 1010 and 0111**, each with probability 0.25.

4.6 Did the measurement agree?

Table 10. *Task 2B results (1000 shots); every observed string is a correct addition*

String	A	B	SUM	CARRY	Theory P	Counts	Measured P
0000	0	0	0	0	0.25	250	0.250
1001	1	0	1	0	0.25	247	0.247
1010	0	1	1	0	0.25	259	0.259
0111	1	1	0	1	0.25	244	0.244

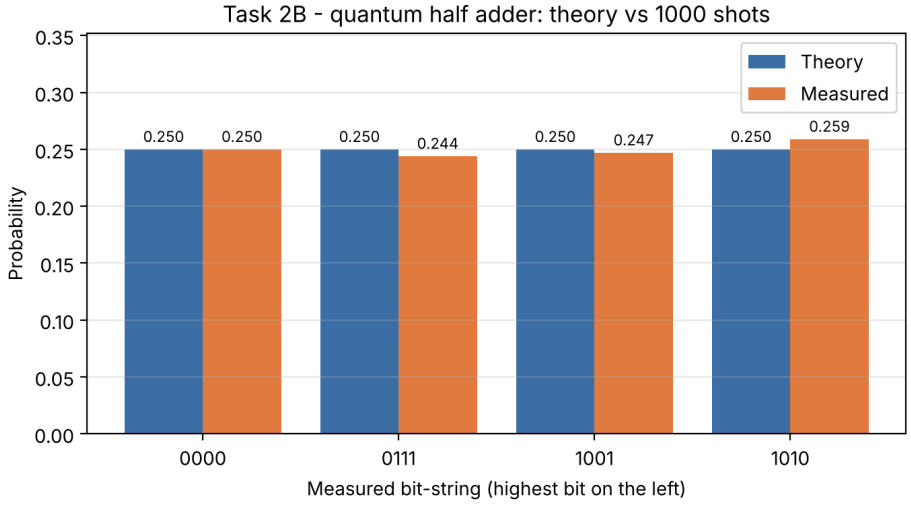


Figure 9. *Task 2B histogram: the four rows of the half-adder truth table*

All **1000 of 1000** shots were correct additions, the four rows appeared 244–259 times (expected 250 ± 13.7), and the TVD is 0.009. The circuit therefore works as designed.

4.7 Can Quokka run every gate in our design?

No, and checking this was an important engineering step. The official *Quokka docs* notebook lists the QASM gates the Quokka simulator understands: **h, x, y, z, rx, ry, rz, cx, swap, cphase and U** (Quokka Computing, n.d.-b). Our design uses **s** and **ccx** (Toffoli), which are not on the list, so the script as written would be rejected with error 7 (INVALID_QASM). Real quantum devices are the same: they run only a small native gate set, and a Toffoli must be decomposed into one- and two-qubit gates (Barenco et al., 1995). The shared helper therefore rewrites every circuit into Quokka's gate set before it is sent: S becomes RZ(π/2), and the Toffoli becomes 6 CNOT, 2 H and 7 RZ(±π/4) gates. The notebook proves that the rewritten circuit has exactly the same probabilities, and the local simulator runs the same rewritten script. The cost is visible: the one-line Toffoli became about 15 gates, and on noisy hardware every extra gate adds error.

4.8 What did Task 2 teach us? (Critical evaluation)

- **Two Bell states, one gate apart.** Adding X before the CNOT changes $|\Phi^+\rangle$ (always equal) into $|\Psi^+\rangle$ (always different). Mathematics predicted this before we ran anything (LO1).
- **Superposition is not free parallel output.** The half adder holds four additions, but each shot shows only one of them. Reading all rows took 1000 shots; a classical half adder needs only 4 runs. A quantum advantage needs interference that concentrates probability on useful answers, as in Grover's or Shor's algorithms, and simple superposition does not do that.
- **Phase matters only with interference.** Z and S changed amplitudes but not one count. They would matter only if a later H gate made the amplitudes interfere.
- **QASM is the common language.** The same OpenQASM 2.0 text ran through Qiskit, the helper and the device interface, which made the work portable between tools (Cross et al., 2017).

5. Task 3 – Investigating a Quantum Code: Quantum Tic-Tac-Toe

> Task question: Complete the provided code by filling in missing instructions or comments; play multiple games and analyse the generated code; as a quantum software engineer, explain the purpose, functionality, gameplay dynamics and how circuits are generated (Word document); and describe each quantum gate used in a separate section. Notebook: Task_3_Quantum_TicTacToe/Task3_Quantum_TicTacToe_Completed.ipynb (original kept unchanged beside it)

5.1 What is the purpose of the program?

The program, supplied as Lab 5 (Bahrami, 2026b), is a **teaching game**. Quantum games have long been used to make superposition and measurement intuitive (Goff, 2006). Here every tile of the 3 × 3 board is **one qubit**, and every move adds a **gate** to a nine-qubit circuit. Nobody owns a tile until the end, when the circuit is measured once: a 0 makes the tile X's and a 1 makes it O's. Playing therefore means *designing a quantum circuit*, and the result is probabilistic. The game makes three ideas visible: superposition (undecided tiles), unitary gates as moves, and measurement collapse.

5.2 How is the code organised?

The code has two classes. **`Board`** is the quantum part: it owns the `QuantumCircuit(9, 9)`, adds gates in `make_move`, measures in `measure`, and counts winning lines in `countWinners`. **`Game`** is the classical user interface: it creates buttons with `ipywidgets`, reacts to clicks in `handle_game`, and redraws the board. This separation of quantum logic from the user interface is good software design. It is also how real quantum applications are built: a classical program prepares and sends circuits and then interprets the results.

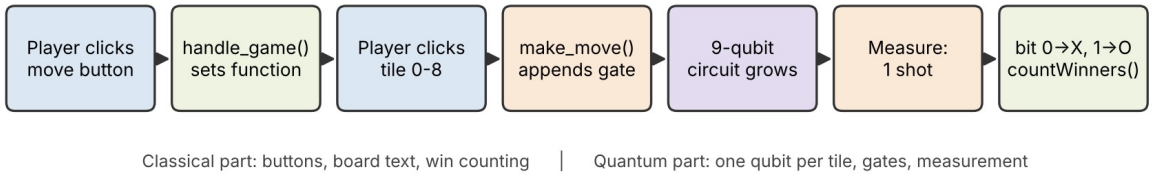


Figure 10. *How one move travels through the code: from a click to a gate, and from measurement to the score*

5.3 What was missing, and what did we complete or fix?

Table 11. *Code completion and fixes (marked [ADDED], [FIXED] or [CHANGED] in the notebook)*

#	Location	Problem in supplied code	Our completion
1	make_move – Not	comment only, no gate	self.qc.x(cell)
2	make_move – O	comment only, no gate	self.qc.ry(+π/4, cell) (toward |1⟩ = O)
3	make_move – X	comment only, no gate	self.qc.ry(−π/4, cell) (toward |0⟩ = X)
4	make_move – SWAP	comment only, no gate	self.qc.swap(cell, self.target)
5	countWinners	winners = () – nobody could ever win	the 8 lines: 3 rows, 3 columns, 2 diagonals
6	SWAP branch	dead inner if; crash if no first tile	guard target ≥ 0; reset target after swap
7	new()	empty tile '' but __init__ uses ' '	made consistent
8	imports / grid	google.colab widgets only work in Colab	ipywidgets HBox/GridBox (VS Code, Jupyter,

#	Location	Problem in supplied code	Our completion
			Colab)
9	screen updates after a click	callback output only shown in Colab; in Jupyter clicks seemed to do nothing	draw every screen inside an ipywidgets Output widget

An important finding: the supplied comments had the rotation directions swapped. `measure()` maps bit 0 to X and bit 1 to O, but the comments said the O button rotates toward $|0\rangle$ and the X button toward $|1\rangle$. Followed literally, pressing **O** would help player **X**. The game's own rules say "O turns the tile toward a tile owned by O", so we implemented O as a rotation toward $|1\rangle$ and X toward $|0\rangle$ and corrected the comments. Finding a mismatch between comments, code and specification, and resolving it against the specification, is a normal part of investigating someone else's code.

5.4 How does a move become a gate? (Circuit generation)

1. **Start of a round.** `__init__` (and `new()`) adds `reset` and **H** to all nine qubits, then a barrier. Every tile is $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$, i.e. 50% X and 50% O.
2. **Choose a move.** Clicking Not, O, X or SWAP raises a `ValueError` when `int(b.description)` is tried, and the `except` branch stores the move in `board.function`. This is a slightly unusual way to tell text buttons from number buttons.
3. **Choose a tile.** Clicking 0–8 calls `make_move(cell)`, which appends one gate to `self.qc`. For SWAP, the first tile click sets `board.target` and the second click applies the swap.
4. **The circuit grows.** After each move the circuit is redrawn. The text on the board is only a move history; as the original notes say, "the game circuit is the true state of the game".
5. **Measure.** Barrier and nine `measure` instructions are added, and the circuit runs **once** (1 shot) on AerSimulator. Bit i of the result decides tile i, read from the right-hand end of Qiskit's string (`output[9-1-i]`).
6. **Score.** `countWinners` checks the eight lines for each player. Both players can win lines in the same round, because the tiles are decided independently.

5.5 How does a rotation move change the odds?

On the Bloch sphere, $|+\rangle$ sits at angle $\theta = \pi/2$ from $|0\rangle$, and $P(X) = P(0) = \cos^2(\theta/2)$. Each X move applies $R_y(-\pi/4)$, which reduces θ by $\pi/4$ (Figure 11). One X move gives $P(X) = \cos^2(\pi/8) = 0.854$, two moves give 1.000, and **a third move over-rotates** back to 0.854. Moves are rotations, not counters, so "too many" moves undo themselves. This is a quantum effect that classical tic-tac-toe does not have.

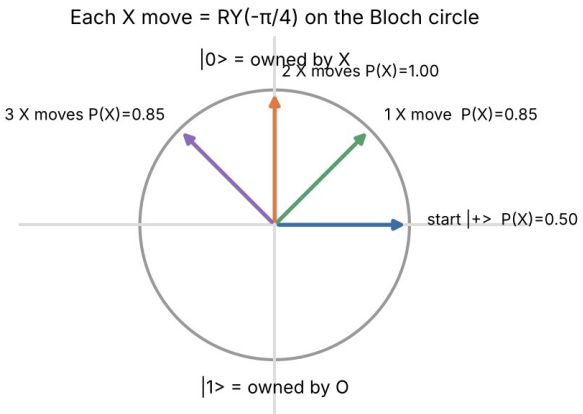


Figure 11. *Effect of repeated X moves on one tile (x–z plane of the Bloch sphere)*

5.6 What happened when we played? (Four recorded games)

Button clicks cannot be saved in a notebook, so we replayed four games as scripts that call exactly the same `make_move` and `measure` code with fixed seeds. For each game we saved the circuit picture, the one-shot result, and a **1000-shot study** of the same circuit, which estimates each tile's chance of becoming X and each player's chance of winning (more X lines than O lines).

Table 12. *Recorded games: moves, generated gates, the one-shot result and 1000-shot win chances*

Game	Moves	Gates added	1-shot lines X:O	X wins /1000	O wins /1000	Draws / 1000
1 Baseline	none	9 H only	4 : 0	392	405	203
2 Diagonals	X on 0, 4, 8; O on 2, 6	+5 RY	4 : 0	705	210	85
3 NOT test	X, X on 4; Not on 4; O on 0; Not on 1	+3 RY, 2 X	0 : 2	135	694	171
4 SWAP	X, X on 0; O, O on 4; SWAP 0 ↔ 8; X × 3 on 2	+7 RY, 1 SWAP	0 : 2	404	289	307

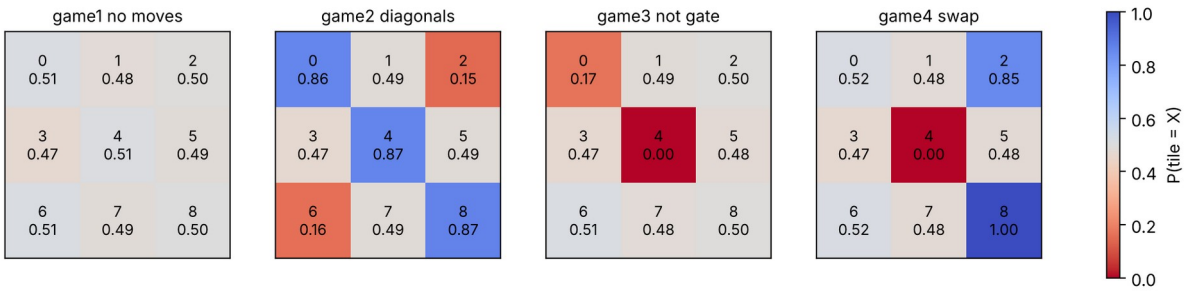


Figure 12. *Chance that each tile becomes X after each recorded game (1000 shots; blue = X likely, red = O likely)*

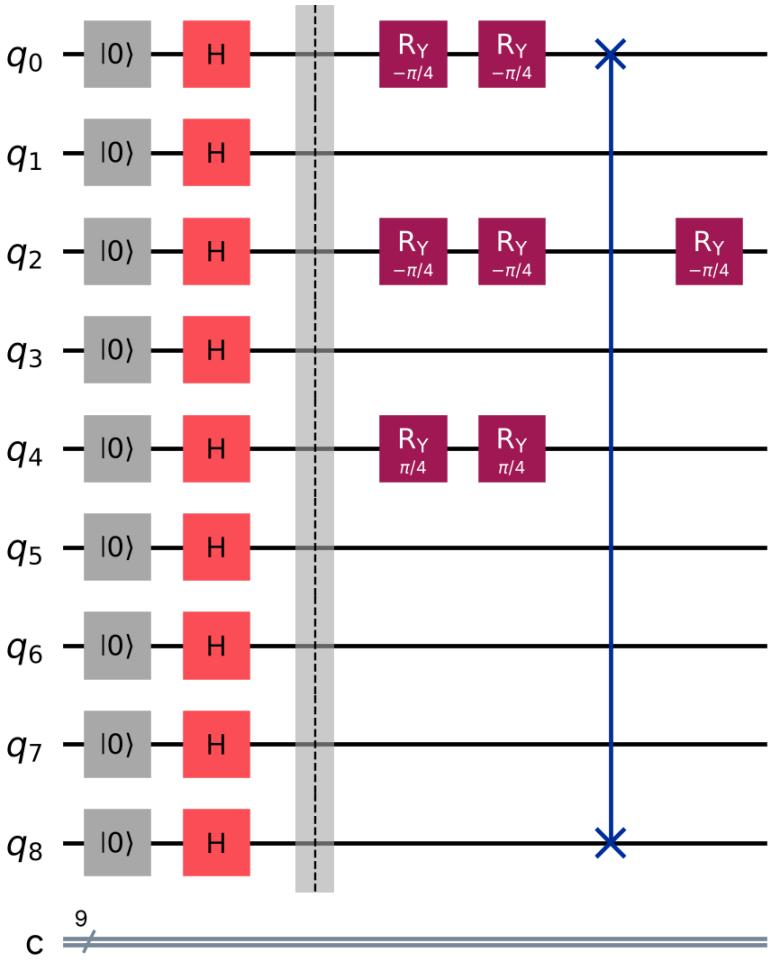


Figure 13. *Circuit generated by Game 4: H start layer, RY moves, and the SWAP between tiles 0 and 8*

5.7 What do the games show? (Analysis)

- **Game 1 (baseline):** with no moves every tile is 50/50, so X and O win about equally often (392 vs 405). The one-shot result (X 4 lines) was simply luck. A single game says little; the 1000-shot study shows the real odds.
- **Game 2:** three X moves on the main diagonal (P(X) ≈ 0.86 each) and two O moves on the other corners raised X's win chance from 39% to 71%. Strategy works, but only in probability.
- **Game 3 (NOT gate):** two X moves made tile 4 certainly X, and NOT then made it certainly O (P(X) = 0.00). NOT on the *undecided* tile 1 did **nothing** (0.49), because $|+\rangle$ is an eigenstate of X: $X|+\rangle = |+\rangle$. This matches the rule "if the tile is not owned, Not does nothing", and it is explained by linear algebra, not by an if-statement.
- **Game 4 (SWAP):** tile 0 was made certainly X and then swapped with tile 8. Afterwards tile 8 is certainly X (1.00) and tile 0 is back to 50/50, because SWAP moved the quantum *states*, not labels. Tile 2 received three X moves and shows the over-rotation (0.85, not 1.00).
- **Circuit growth:** every move adds one gate and does not remove any, so circuit depth rose from 2 to 5. On noisy hardware, longer games would therefore be less reliable.

5.8 Did we also play the game by clicking the buttons?

Yes. To check that the completed game really works for a player, we opened the notebook in **Jupyter Notebook 7** (the environment used in the lectures), ran it, and played three games by clicking the move and tile buttons. Each **Measure** click runs the game circuit once, so every result is a genuine single random outcome.

Table 13. *Three games played interactively in Jupyter Notebook (31 October 2026)*

Game	Buttons clicked	Final board (rows)	Result and what it shows
A	X 4, X 4, O 0, O 8, Not 2	X O O / O X X / O X O	Draw 0–0. Tile 4 (two X moves) was certain X. Tile 0 had one O move (85% O) yet became X: a 15% outcome happened.
B	X 0, X 4, X 8, O 2, O 6	X O O / X X O / O O X	X wins the main diagonal 0–4–8, the three tiles nudged toward X.
C	O 4, O 4, X 1, Not 4, SWAP 0 ↔ 8	X X X / X X O / O O X	X wins 2 lines. Two O moves made tile 4 certain O, then NOT made it certain X, exactly as the theory predicts.

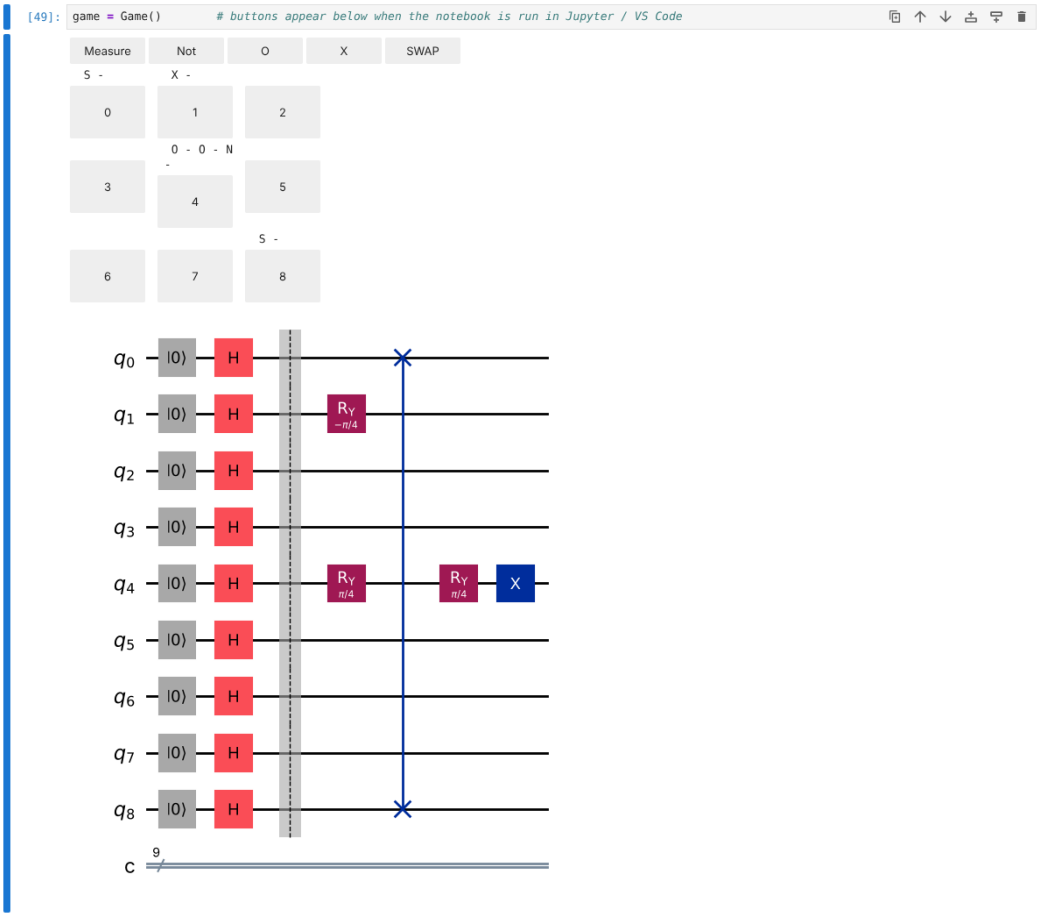


Figure 14. *Game C in Jupyter before Measure: the board shows the move history, and the circuit shows RY, X and the SWAP between tiles 0 and 8*

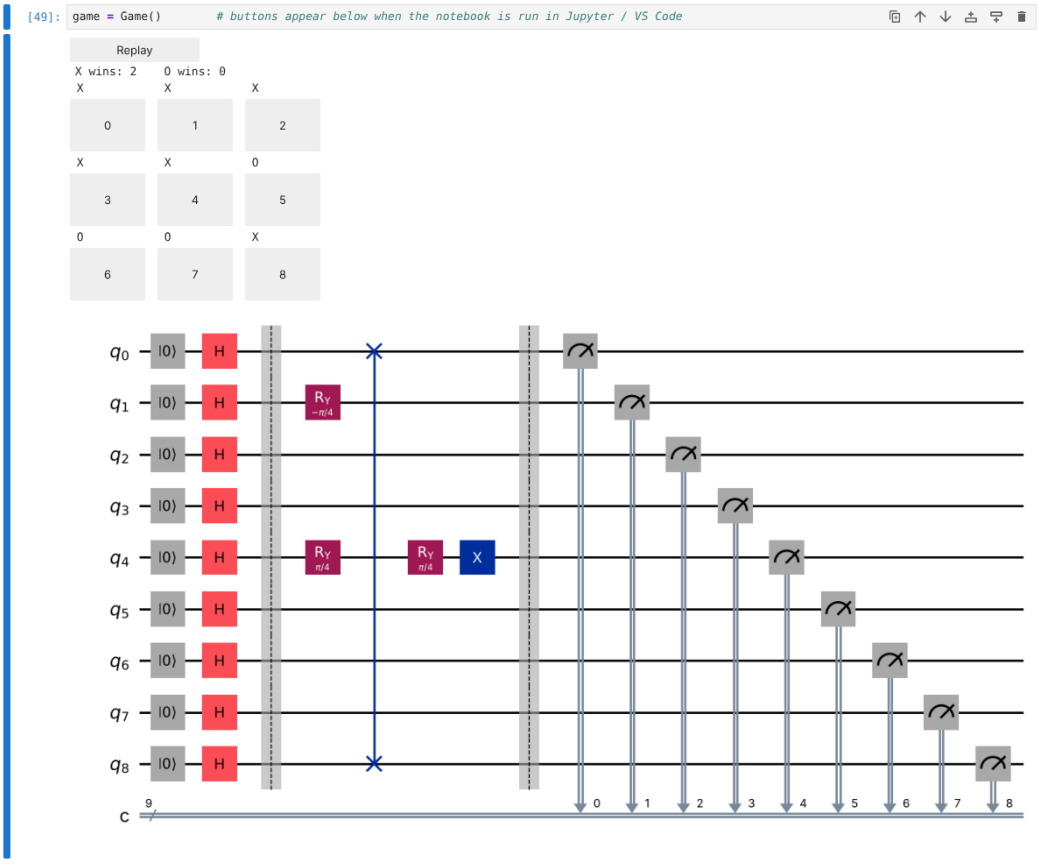


Figure 15. *Game C after Measure: one shot decided every tile; X wins 2 lines*

Playing also exposed a **portability bug**. In the supplied code, every screen update after a click (board, circuit, score) is printed from inside the button's callback. Google Colab shows that output, but Jupyter Notebook and VS Code discard it, so clicks *appeared to do nothing*. We fixed this by drawing all game screens inside an `ipywidgets.Output` widget (marked `[CHANGED]`), after which the game worked in all three environments.

5.9 Can the game circuit run on Quokka?

Yes. The notebook removes the `reset` instructions (qubits already start at $|0\rangle$), translates the Game 2 circuit to H, RY, X and CX gates, exports it to QASM and sends it with 1000 shots. With the labelled local fallback, the per-tile chances were 0.87, 0.50, 0.13, 0.51, 0.86, 0.50, 0.14, 0.53 and 0.87, the same pattern as Figure 12. This links the game back to the device used in Tasks 1 and 2.

5.10 Which quantum gates does the game use? (Gate section)

Table 14. *Quantum gates used in circuit generation: characteristics and function in the game*

Gate	Matrix / action	Key characteristic	Function in this game
H (Hadamard)	$(1/\sqrt{2})[[1,1],[1,-1]]$; $\|0\rangle \to \|+\rangle$	creates equal superposition; H·H = I	start layer: every tile undecided (50/50)
$R_y(\theta)$	[[cos θ/2, −sin θ/2],[sin θ/2, cos θ/2]]	rotation about the y-axis; real amplitudes; changes probabilities smoothly	X move: $R_y(-\pi/4)$ toward $\|0\rangle$; O move: $R_y(+\pi/4)$ toward $\|1\rangle$
X (NOT)	[[0,1],[1,0]]; $\|0\rangle \leftrightarrow \|1\rangle$	bit flip; $X\|+\rangle = \|+\rangle$ (eigenstate)	Not move: flips ownership of a decided tile
SWAP	exchanges two qubits' states (= 3	two-qubit, permutes $\|01\rangle$	SWAP move: exchanges two tiles

Gate	Matrix / action	Key characteristic	Function in this game
	CNOTs)	↔ |10⟩	
reset	sets qubit to |0⟩ (not unitary)	non-reversible initialisation	clean start of each round
measure	projects onto |0⟩ or |1⟩	irreversible; Born rule	decides each tile's owner (0 → X, 1 → O)
barrier	no operation	visual / compiler boundary	separates start, moves and measurement in the drawing

5.11 How do these gates fit the wider theory of quantum logic gates?

Every quantum gate is a **unitary matrix** ($U^\dagger U = I$), so every gate is **reversible**: applying $U^\dagger$ undoes it. `reset` and `measure` in the table above are the exceptions; they are not gates in this strict sense because they cannot be undone, and measurement follows the Born rule (Nielsen & Chuang, 2010; "Quantum logic gate," n.d.). Gates in sequence combine by **matrix multiplication**, written in the reverse of the drawing order (X then Y gives the matrix YX), and gates on different qubits combine by the **tensor product**. That is how the nine-qubit game state is built from nine single-qubit operations.

A gate set is **universal** if it can approximate any unitary operation; examples are {H, T, CNOT}, the Clifford+T set, or arbitrary rotations plus CNOT (Barenco et al., 1995; "Quantum logic gate," n.d.). This idea explains two of our findings:

- **The game's gates are not universal and cannot entangle tiles.** H, R_y and X keep every amplitude real, and SWAP only moves states between tiles. The game has no entangling gate such as CNOT, so its nine tiles always stay independent. Adding a CNOT move ("link two tiles") would make the gate set universal and allow entangled tiles.
- **Quokka's gate set is universal.** Its QASM gates include Rx, Ry, Rz and CNOT, which form a universal set. That is why `to_quokka_gates()` can always rewrite S, T or the Toffoli into supported gates (Task 2B). The rewritten circuit can differ only by a **global phase**, which never changes a measurement result.

5.12 What are the strengths and limits of the program?

Strengths: the program is short, has clear quantum/classical separation and a good visual circuit, and it demonstrates superposition, rotation and measurement in a playful way. **Limitations:** (1) tiles are never *entangled*, because the only two-qubit gate is SWAP, which keeps tiles independent; adding a CNOT move would create linked tiles and is a natural improvement. (2) One-shot measurement makes single games very random. (3) The board text is "not 100% accurate", so it can mislead players. (4) Using exceptions to tell buttons apart is fragile. (5) There is no turn order, so one player can make every move. Despite these limits, as a teaching tool it shows clearly that a quantum program is a circuit that is built step by step and only gives answers when measured.

6. Task 4 – Machine Learning Quantum Analysis

> Task question: Analyse the code (circuit and problem formulation; data input; quantum functionality, excluding the optimiser); execute it, monitor the output at each iteration and plot iteration vs output and iteration vs time; modify it to run locally without the quantum computer or circuit and document differences in efficiency and effectiveness; submit the modified .ipynb and a report. Notebook: Task_4_Quantum_ML/Task4_Quantum_ML_Modified.ipynb (original kept beside it)

6.1 What problem does the code solve? (Problem formulation)

The supplied Lab 10 notebook (Bahrami, 2026c) trains a quantum circuit to classify tiny 2 × 2 black-and-white images into two classes, which the code calls **stripes** (label 0) and **bars** (label 1). `generate_data(2)` creates all line patterns and removes the all-black and all-white images, because they belong to both classes. Four images remain: [1,0,1,0] and [0,1,0,1] (label 0), and [1,1,0,0] and [0,0,1,1] (label 1). Three are used for training and one for testing (75/25 split).

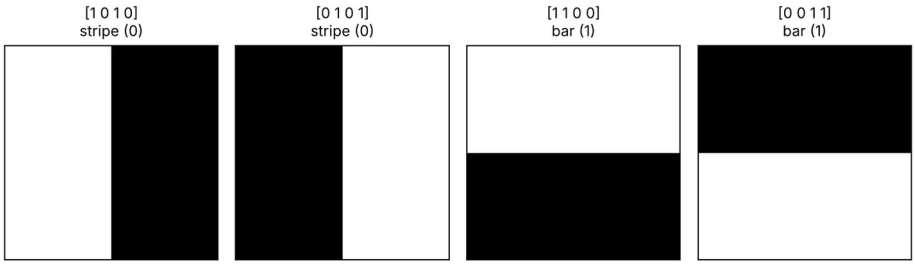


Figure 16. *The four images, their pixel vectors and labels*

The model output is the probability that the last qubit is measured as 1, estimated from shots: $\hat{y}(\theta, x) \approx$ (number of 1s) / shots. The **loss** minimised is the mean absolute error over the training images:

$$L(\theta) = (1/N)\ \Sigma_n \mid \hat{y}(\theta, x_n) - y_n \mid$$

A good model gives $\hat{y} \to 1$ for bars and $\hat{y} \to 0$ for stripes; an image is classified as a bar when $\hat{y} > 0.5$.

6.2 Which part of the code is the input to the circuit?

The input is the loop at the top of `create_circuit`: for every pixel i, `if data[i] > 0.5`, an **X gate** is added to qubit i. Each pixel is one qubit, and a white pixel flips its qubit from $|0\rangle$ to $|1\rangle$. This is **basis encoding**: the image becomes the starting basis state, e.g. [1,0,1,0] → $|q3q2q1q0\rangle = |0101\rangle$ (Schuld & Petruccione, 2021). The data enters only here; everything after it is the trainable model.

6.3 How does the circuit work? (Quantum part, excluding the optimiser)

The trainable part is built from **blocks**. `block(angles, qubits)` applies $R_Y(\theta_a)$ to one qubit, $R_Y(\theta_b)$ to another, and then a CNOT between them. `generate_pairs` splits the qubits in half again and again and joins the halves, giving the pairs (0,1), (2,3) and then (1,3). Information from all four pixels therefore flows into **qubit 3** in only $\log_2 4 = 2$ layers. This is a **tree tensor network (TTN)** classifier of the kind studied by Grant et al. (2018) and Huggins et al. (2019). With 3 blocks × 2 angles there are **6 trainable parameters**, and only qubit 3 is measured.

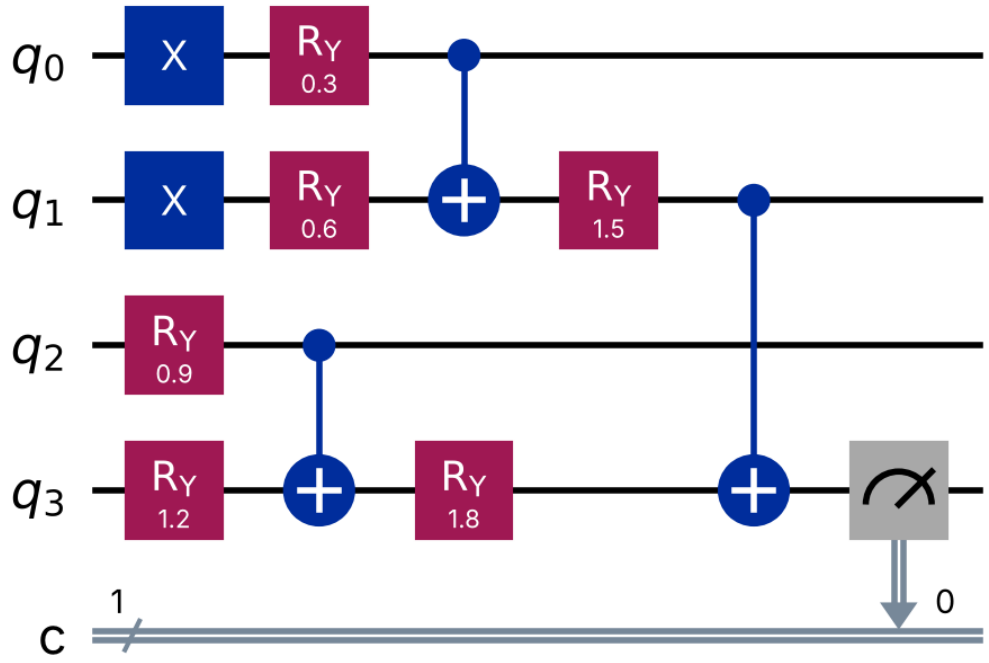


Figure 17. *Classifier circuit for image [1,1,0,0]: X gates encode the pixels, then three RY–RY–CNOT blocks form a tree into q3*

The RY gates give each block adjustable 'weights', while the CNOTs make the output depend on *combinations* of pixels. If a control qubit is in |1⟩, the CNOT flips the target, so the final value of q3 behaves like an XOR (parity) of pixels that is smoothly adjusted by the angles. This structure turns out to be the key to the results below.

6.4 What did we add to monitor every iteration?

We modified the SPSA optimiser (Spall, 1992) without changing its algorithm. SPSA estimates the gradient from just **two** loss evaluations per iteration, whatever the number of parameters, which suits noisy, expensive quantum evaluations. We added a timer around each update and, after each update, one extra evaluation of the training loss, the **output measure**. This monitoring evaluation is timed separately so it does not inflate the iteration time. Settings were kept as in the original ($a = 1$, $c = 0.5$, $\alpha = 0.602$, $\gamma = 0.101$, 100 iterations), except that the shots per circuit were raised from 100 to the group's agreed **1000**, and a fixed seed (2026) makes runs repeatable. Each iteration runs 3 evaluations × 3 training images = 9 circuits, so the run executed **900 circuits** in total.

6.5 What do the two required graphs show?

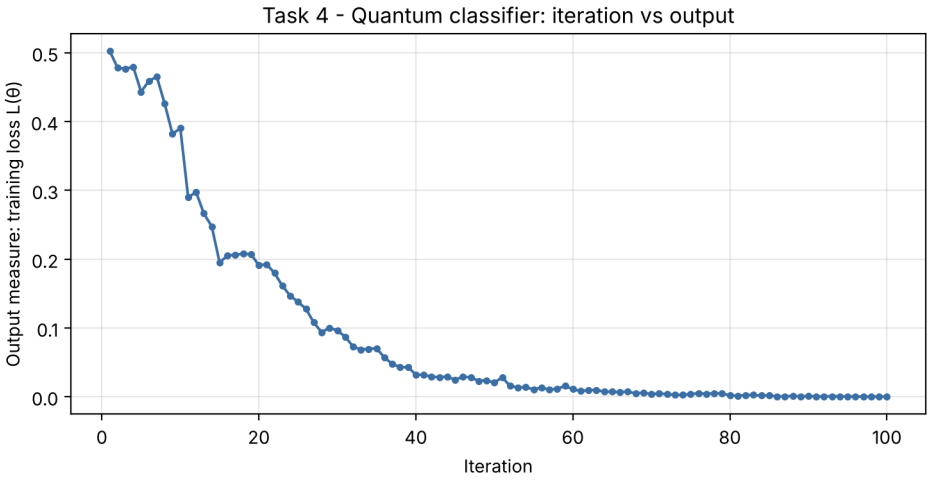


Figure 18. *Required graph 1: iteration number versus output measure (training loss)*

Table 15. *Excerpt from the 100-iteration log (task4_quantum_iterations.csv)*

Iteration	Output (training loss)	Note
1	0.503	random start: no better than guessing
10	0.390	
20	0.191	
30	0.096	
37	< 0.05	first iteration below 0.05
50	0.020	
70	0.004	
100	0.000	all training images perfectly separated

Graph 1 shows the loss falling from 0.50, which is equal to random guessing, to 0.00 by iteration 100. It fell quickly in the first 40 iterations and then flattened. The curve is not perfectly smooth. Each loss value is itself an estimate from 1000 shots per image, and SPSA's random ±1 directions sometimes give a slightly worse step (e.g. iterations 10–11 and 15–20). The step size $a_k = 1/(k+1)^{0.602}$ shrinks over time, which explains the slow final convergence.



Figure 19. *Required graph 2: iteration number versus time per iteration*

Graph 2 shows an almost **constant** time per iteration of 0.61 s on average (range 0.51–0.72 s). This is expected: every iteration runs the same number of circuits (6 for the update) with the same size and shot count, so the cost does not depend on how good the model is. The small spikes are background noise from the operating system. On the real Quokka, each circuit adds a network round trip, so the line would sit higher and be more irregular, and the total would be dominated by communication rather than computation.

6.6 How did we make the code run without the quantum computer or circuit?

The brief can be read in two ways, so we built **both** and added one fairer baseline. Every version uses the same loss, the same SPSA settings and the same random seed, so only the model changes:

- **Local-A: no quantum computer.** The same circuit is simulated exactly with our own NumPy state-vector code: 16 complex amplitudes, with gates applied as matrices. It computes P(q3 = 1) exactly, so there is no shot noise. It was verified against Qiskit (both gave 0.500625 for a random test case).
- **Local-B: no quantum circuit.** A classical logistic regression $\hat{y} = \sigma(w \cdot x + b)$ with 5 parameters, close to the circuit's 6.
- **Local-C: a fair classical baseline.** Logistic regression with pairwise pixel products added (4 + 6 features, 11 parameters). This is the standard classical way to model interactions.

6.7 What were the differences in efficiency and effectiveness?

Table 16. *Quantum circuit versus local versions (100 SPSA iterations, seed 2026). Accuracy uses each trained model's exact output; times vary slightly between runs*

Version	Total time (s)	Time / iteration (s)	Circuit runs	Final loss	Test acc.	All 4 images
Quantum circuit (1000 shots; local sim, Quokka offline)	91.7	0.614	900	0.000	1/1	4/4
Local-A: same circuit, exact NumPy	0.19	0.0013	0	0.0003	1/1	4/4
Local-B: classical linear	0.024	0.00016	0	0.296	0/1	3/4
Local-C: classical + pairwise features	0.034	0.00023	0	0.206	0/1	3/4

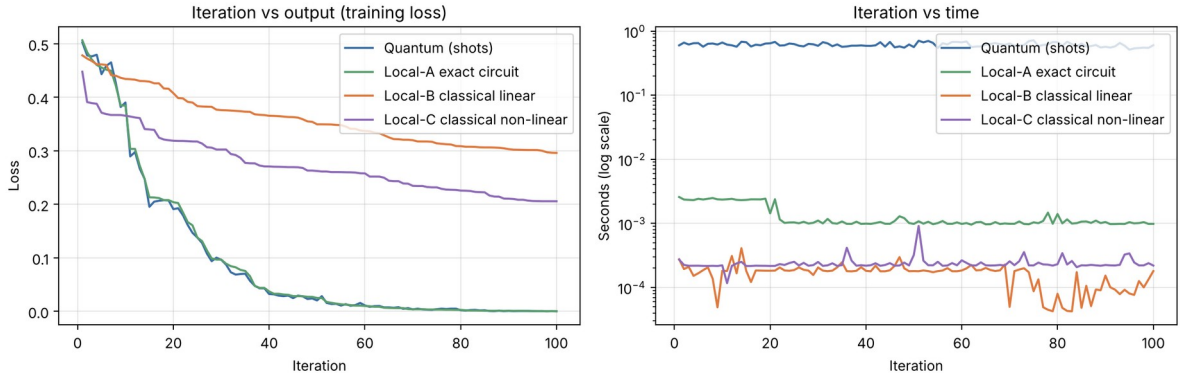


Figure 20. *All versions: loss per iteration (left) and time per iteration on a log scale (right)*

Efficiency. Removing the quantum device made each iteration about **470 times faster** (0.614 s → 0.0013 s). A shot-based run needs 1000 samples per circuit, plus QASM parsing and, on Quokka, a network request, whereas the exact simulation needs one small matrix calculation. Local-A and the quantum run followed almost the same loss curve, both first dropping below 0.05 at iteration 37. The shot noise therefore slowed the computer, not the learning. This advantage disappears as problems grow: exact simulation needs 2^n amplitudes, so 50 qubits would need about 10^{15} complex numbers, which is why real quantum hardware becomes useful only at scale.

Effectiveness. The circuit (both quantum and Local-A) reached zero training loss and also classified the **unseen test image** correctly. Both classical models fitted the training images but **failed the test image**, and their losses stayed high (0.30 and 0.21). The reason is the shape of the data. In each class the two images are exact opposites, so the class sums are identical and **no straight line can separate the classes**. Like XOR, this is the classic limit of linear models (Minsky & Papert, 1969). In fact, the label equals **x_1 XOR x_3** (Figure 21).

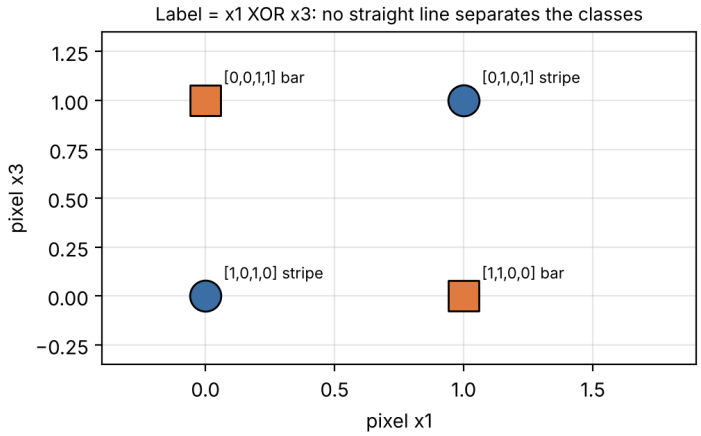


Figure 21. *The data is XOR-shaped: label = x1 XOR x3, so a linear classifier cannot separate it*

Local-C has the right *kind* of features, but the test image's key feature (the product $x_0 \cdot x_1$) is zero in all three training images, so its weight is never learned. The circuit generalised because its CNOT tree computes parity naturally. Its built-in bias (an XOR-like structure) happens to match this problem. This is an honest, limited claim: with one test image the test score is weak evidence, and a classical model designed with XOR features would also succeed. The fair conclusion is that the circuit's **structure**, not quantum speed, made it effective here.

6.8 Did anything not work as expected?

Yes. The original warm-up (one qubit learning to output 1 with Nelder–Mead, 10 iterations) **failed** in our run: it stopped at $\theta = -0.47$ with loss 0.87 instead of $\theta = \pi$. Near $\theta = 0$ the loss $\cos^4(\theta/2)$ is almost flat, and differences between nearby points are smaller than the shot noise, so the simplex method (Nelder & Mead, 1965) could not tell which way to move in 10 iterations. This small failure illustrates a real problem in quantum machine learning: noise plus flat loss landscapes make training hard, and in large circuits this becomes the 'barren plateau' problem (McClean et al., 2018). SPSA was more robust because it uses large, random perturbations.

6.9 What is our overall judgement for Task 4?

- **Input:** pixels → X gates (basis encoding) in `create_circuit`; **model:** 6 RY angles in a TTN of RY–RY–CNOT blocks; **output:** P(q3 = 1).
- **Monitoring:** the loss fell from 0.50 to 0.00 in 100 iterations; time per iteration was flat at about 0.6 s.
- **Efficiency:** at 4 qubits, simulation without the device is about 470 times faster; real devices only pay off where simulation is impossible.
- **Effectiveness:** the circuit solved an XOR-type problem that linear classical models could not. This comes from its CNOT structure, not from a proven quantum advantage (Benedetti et al., 2019).

7. Critical Discussion Across the Four Tasks

7.1 How does the evidence meet each learning outcome?

Table 17. *Evidence for each learning outcome*

LO	Requirement	Where this report shows it
LO1	Critically evaluate quantum protocols with the maths of quantum mechanics to validate performance claims	State evolution tables (Tables 5, 7); TVD and z-scores; the Z/S phase lesson; the eigenstate explanation of NOT; the XOR analysis and the limits of the Task 4 'advantage'
LO2	Design and create quantum circuits for quantum algorithms to run on quantum computers	Cirq Bell circuit, Qiskit H–X–CNOT, original QASM half adder with an automatic Toffoli decomposition, completed game gates, QASM export of a game circuit, the ML circuit run through the device interface
LO4	Communicate application and critical analysis to software-industry audiences	WH-question structure, data-flow and pipeline diagrams, GitHub README with Colab instructions, maintenance guide, labelled evidence files and an honest offline note

7.2 What are the main limitations of our work?

- **Quokka availability.** The results in this draft came from the labelled local simulator because the Quokka endpoint was offline. The code is ready, and `run_all.bat` repeats every experiment on Quokka with one click.
- **An ideal device hides noise.** Quokka emulates a fault-tolerant machine, so our experiments cannot show decoherence or gate errors. A future step would be to add a Qiskit noise model, or a real noisy backend, and repeat Tasks 1 and 2 to see 01/10 leakage.
- **Small data in Task 4.** Four images and one test image are too few for strong conclusions about generalisation.
- **Mostly simulated gameplay evidence in Task 3.** Three games were played by clicking; the four detailed studies were replayed by script so they can be repeated exactly.

7.3 What would we do next?

With more time we would: (1) re-run all tasks on Quokka and compare the device with the simulator using the TVD; (2) add an entangling CNOT move to the tic-tac-toe game so that tiles can be linked; (3) test the Task 4 classifier on larger images ($3 \times 3 = 9$ qubits) to see when exact simulation slows down; and (4) add a noise model to measure how fidelity falls as circuits get deeper.

8. Development Process and Code Walkthrough

This section explains how the project was built, step by step, and what every part of the code does, so that a reader can follow, re-run and change it. Every code cell in the notebooks also carries inline comments.

8.1 What process did we follow?

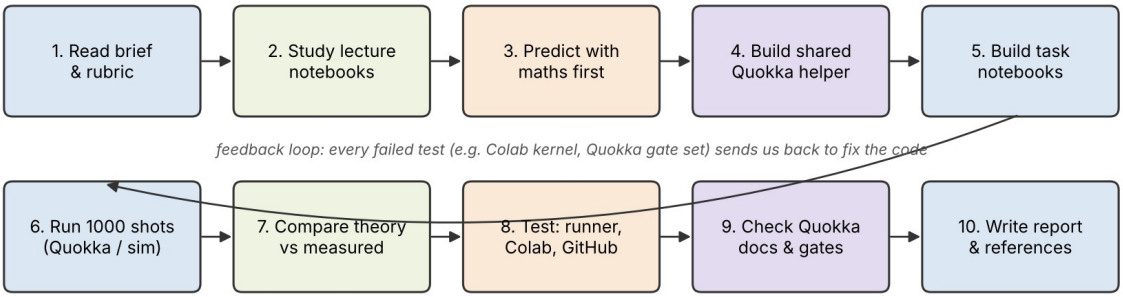


Figure 22. *Development process used for every task, with a feedback loop after each test*

1. **Read the brief and the rubric.** List what each task must show (e.g. "executed on the Quokka device", "Word document", "plots") and turn each item into a checklist.
2. **Study the lecture material.** The Week 4/7 `quokka_access` notebook gave the device address and request format; the Week 12 lab notebooks are the supplied code for Tasks 3 and 4.
3. **Predict with mathematics first.** Before running a circuit we wrote down its expected state and probabilities (state-evolution tables in Sections 3–4). This turns each run into a test.
4. **Build one shared helper** (`common/quokka_helper.py`) so that every task sends circuits the same way, with the same 1000 shots and the same evidence files.
5. **Build each task notebook** in the same order: objective → theory → circuit → code → run → theory-vs-measured table and chart → discussion.
6. **Run with 1000 shots** on Quokka, or on the labelled local simulator when Quokka is offline. The simulator uses a fixed seed, so every result can be repeated exactly.
7. **Compare theory with measurement** using the TVD and z-score, and explain every difference.
8. **Test the whole project end to end** with `run_project.py`, in Google Colab and from GitHub. Two real problems were found this way and fixed (Section 8.4).
9. **Check the device documentation.** The Quokka docs revealed its limited gate set, which led to automatic gate translation.
10. **Write the report** from the saved results, and verify every reference against Crossref and DataCite.

8.2 How does the shared helper work?

`common/quokka_helper.py` is the only file that talks to Quokka. Each notebook imports it as `qh` and normally calls just one function, `qh.run_qasm(qasm, shots=1000, label=..., save_dir=...)`.

Table 18. *Functions in common/quokka_helper.py*

Function	What it does	Why we need it
_quokka_url()	builds the address from QUOKKA_URL, QUOKKA_NAME or quokka_config.json (default quokka1)	change the device without editing code
clean_qasm(text)	removes include "qelib1.inc", comments, blank lines and barriers	Quokka has no qelib1 library and rejects extra lines
to_quokka_gates(text)	rewrites any gate outside {h, x, y, z, rx, ry, rz, cx, swap} using Qiskit's transpiler	Quokka's documented gate set has no s, t or ccx
_numeric_angles(text)	writes rz(pi/2) as rz(1.5707963...)	the docs only show numeric angles
run_on_quokka(text, shots)	POST {script, count} with verify=False, as in the official notebooks; raises a clear error if offline	the single place that contacts the device
run_locally(text, shots)	runs the same rewritten script on Qiskit Aer with a seed from a fixed sequence	repeatable, honest fallback
shots_to_counts / counts_to_probs	turns [[c0, c1], ...] into {'c1c0': n} and probabilities	one bit order (Qiskit's) everywhere
total_variation_distance(p, q)	½ Σ |p − q|	one number for theory-vs-measurement agreement
run_qasm(...)	tries Quokka, falls back, times the run, saves JSON with source label	the function every task calls
quokka_status()	sends a 1-qubit test script	used by run_project.py status and the watcher
plot_comparison(series, ...)	grouped bar chart with value labels	the same chart style in every task

```
def run_qasm(qasm_text, shots=1000, label="run", save_dir=None,
             allow_fallback=True):
    start = time.perf_counter()              # 1. start the stopwatch
    try:
        data = run_on_quokka(qasm_text, shots) # 2. Quokka first (translated)
        shot_list = list(data["result"].values())[0]
        source = SOURCE_QUOKKA
    except Exception as exc:                 # 3. offline -> same script on Aer
        shot_list = run_locally(qasm_text, shots)
        source = SOURCE_LOCAL
    counts = shots_to_counts(shot_list)      # 4. e.g. {'00': 508, '11': 492}
    ...                                      # 5. save JSON evidence
```

8.3 What does each notebook cell do?

Table 19. *Cell-by-cell guide to the four task notebooks*

Notebook	Cells (in order)	What the code does
All four	1. Title 2. Colab bootstrap	In Colab only: clone the GitHub repository, pip-install Cirq/Qiskit, move into the task folder. Skipped on a PC.
Task 1	3. Imports 4. Build circuit	LineQubit(0..1); H(q0); CNOT(q0, q1); measure into key 'c'.
	5. Theory 6. QASM	cirq.Simulator gives exact amplitudes (0.7071);

Notebook	Cells (in order)	What the code does
		cirq.qasm() exports OpenQASM 2.0; clean_qasm() prepares it.
	7. Run 8–10. Analysis	run_qasm with 1000 shots; results table with difference column; histogram; binomial z-score.
Task 2	A3–A4. Build + theory	QuantumCircuit(2,2): h(0), x(1), cx(0,1) (block C), measure; Statevector gives $\|\Psi^+\rangle$ probabilities.
	A5–A7. Run + compare	Both-qubit and output-only runs; table, TVD, chart.
	B1–B2. Custom QASM	Load custom_half_adder.qasm (h, z, s, ccx, cx); draw; exact probabilities and phases.
	B3. Quokka gates + run	to_quokka_gates() rewrites s and ccx; checks probabilities are unchanged (True); 1000-shot run.
	B4. Truth table	Decodes each string into A, B, SUM, CARRY and checks every shot is a correct addition.
Task 3	Board class	9 qubits; H start; make_move adds x / ry(±π/4) / swap; measure 1 shot; countWinners with 8 lines.
	Game class	ipywidgets buttons and grid; handle_game routes clicks; unchanged rules.
	Recorded games	play_scripted() replays 4 games through the same Board code; 1000-shot study per game; heat map.
	Quokka export	removes reset, translates to h/ry/x/cx, sends Game 2 with 1000 shots.
Task 4	Data + split	generate_data(2) → 4 images; fixed-seed 75/25 split.
	Circuit	create_circuit: X gates encode pixels (input); 3 RY–RY–CNOT blocks in a tree; measure q3.
	SPSA + logging	Two loss evaluations per step; records output and time every iteration; 900 circuits.
	Local versions	Local-A NumPy state vector (checked against Qiskit); Local-B and Local-C classical models; comparison table and plots.

8.4 What went wrong during development, and how did we fix it?

Table 20. *Problems found by testing, and their fixes*

Problem	Cause	Fix
Quokka returned HTTP 404	device server offline (ERR_NGROK_3200)	labelled local fallback; status test; watcher; configurable address
Bell counts changed every run	unseeded local simulator	fixed seed sequence (LOCAL_SEED = 802) → repeatable evidence
Colab: No module named 'cirq'	nbconvert started a kernel from a different Python	run_project.py installs missing packages and registers its own kernel
Custom circuit would be rejected by Quokka	s and ccx are not in Quokka's QASM gate list	to_quokka_gates() rewrites them; equivalence checked

Problem	Cause	Fix
Tic-tac-toe could never have a winner	winners = () in supplied code	added the 8 winning lines
O button helped player X	comments swapped rotation directions	followed the game rules; corrected comments
Game failed outside Colab	google.colab.widgets only exists in Colab	standard ipywidgets layout
Clicks did nothing in Jupyter	callback output is discarded outside Colab	game screens drawn inside an Output widget; found by playing in Jupyter
Batch files failed on Windows	Unix line endings break batch labels	saved with CRLF; .gitattributes enforces it

8.5 How did the code change over time? (Version history)

All work is version-controlled in the GitHub repository. Each commit message explains what changed and why, so the history is part of the documentation.

Table 21. *Commit history of github.com/anishnz/quantumcomputing_YB_Anish*

Commit	Date	What changed and why
5025e0c	31 Oct 2026 01:00	First version: four task notebooks, shared helper, runner, Colab notebook, alternative code, report and README.
7bb1d43	31 Oct 2026 01:07	Fix Colab run: the runner installs missing packages and uses its own Jupyter kernel.
47b49f6	31 Oct 2026 01:15	Report: Colab test-run evidence; README: Copy to Drive steps.
501d72d	31 Oct 2026 01:41	Use Quokka's documented gate set: circuits are translated automatically before sending.
c7eab4a	31 Oct 2026 02:10	Configurable Quokka address, connection diagnosis, verified references, this walkthrough, Task 3 and Task 4 Word documents, final ZIP.
(this version)	31 Oct 2026	Block C confirmed as CNOT; game fixed for Jupyter (Output widget); three games played by clicking, with screenshots.

9. Conclusion

The project answered all four tasks with working, commented and repeatable code. **Task 1** prepared the Bell state $|\Phi^+\rangle$ in Cirq and confirmed perfect correlation in 1000 shots. **Task 2** built the given circuit in Qiskit, identified block C as a CNOT and showed it creates $|\Psi^+\rangle$. Our original QASM half adder produced a correct addition in every one of 1000 shots. **Task 3** completed and debugged the quantum tic-tac-toe game, explained how each click becomes a gate, and used four recorded games to show superposition, over-rotation, the eigenstate behaviour of NOT, and SWAP. **Task 4** explained the data encoding and the tree-shaped circuit of a variational classifier, logged and plotted all 100 iterations, and compared the circuit with three local versions. The local versions were about 470 times faster, but only the circuit solved the XOR-shaped data. Across the project, the most important lesson was to **predict with mathematics first, then measure, then explain the difference**. That habit turned code that runs into evidence that convinces.

References

Bahrami, H. (2026a). *From quantum gates to quantum algorithms: Deutsch's algorithm* [Lecture slides, Week 7]. MSE802 Quantum Computing, Yoobee College of Creative Innovation.

Bahrami, H. (2026b). *Lab 5 — Quantum tic-tac-toe* [Unpublished Jupyter notebook, Quantum_Tic_Tac_Toe__AS2.ipynb]. MSE802 Quantum Computing, Yoobee College of Creative Innovation.

Bahrami, H. (2026c). *Lab 10 — Quantum ML* [Unpublished Jupyter notebook, Quantum_ML_AS2.ipynb]. MSE802 Quantum Computing, Yoobee College of Creative Innovation.

Bahrami, H. (2026d). *Quokka access* [Unpublished Jupyter notebook, quokka_access.ipynb, Week 4]. MSE802 Quantum Computing, Yoobee College of Creative Innovation.

Barenco, A., Bennett, C. H., Cleve, R., DiVincenzo, D. P., Margolus, N., Shor, P., Sleator, T., Smolin, J. A., & Weinfurter, H. (1995). Elementary gates for quantum computation. *Physical Review A*, *52*(5), 3457–3467. https://doi.org/10.1103/PhysRevA.52.3457

Bell, J. S. (1964). On the Einstein Podolsky Rosen paradox. *Physics Physique Fizika*, *1*(3), 195–200. https://doi.org/10.1103/PhysicsPhysiqueFizika.1.195

Benedetti, M., Lloyd, E., Sack, S., & Fiorentini, M. (2019). Parameterized quantum circuits as machine learning models. *Quantum Science and Technology*, *4*(4), Article 043001. https://doi.org/10.1088/2058-9565/ab4eb5

Cirq Developers. (2026). *Cirq* (Version 1.7.0) [Computer software]. Zenodo. https://doi.org/10.5281/zenodo.4062499

Cross, A. W., Bishop, L. S., Smolin, J. A., & Gambetta, J. M. (2017). *Open quantum assembly language* (arXiv:1707.03429). arXiv. https://doi.org/10.48550/arXiv.1707.03429

Einstein, A., Podolsky, B., & Rosen, N. (1935). Can quantum-mechanical description of physical reality be considered complete? *Physical Review*, *47*(10), 777–780. https://doi.org/10.1103/PhysRev.47.777

Goff, A. (2006). Quantum tic-tac-toe: A teaching metaphor for superposition in quantum mechanics. *American Journal of Physics*, *74*(11), 962–973. https://doi.org/10.1119/1.2213635

Grant, E., Benedetti, M., Cao, S., Hallam, A., Lockhart, J., Stojevic, V., Green, A. G., & Severini, S. (2018). Hierarchical quantum classifiers. *npj Quantum Information*, *4*, Article 65. https://doi.org/10.1038/s41534-018-0116-9

Hensen, B., Bernien, H., Dréau, A. E., Reiserer, A., Kalb, N., Blok, M. S., Ruitenberg, J., Vermeulen, R. F. L., Schouten, R. N., Abellán, C., Amaya, W., Pruneri, V., Mitchell, M. W., Markham, M., Twitchen, D. J., Elkouss, D., Wehner, S., Taminiau, T. H., & Hanson, R. (2015). Loophole-free Bell inequality violation using electron spins separated by 1.3 kilometres. *Nature*, *526*(7575), 682–686. https://doi.org/10.1038/nature15759

Huggins, W., Patil, P., Mitchell, B., Whaley, K. B., & Stoudenmire, E. M. (2019). Towards quantum machine learning with tensor networks. *Quantum Science and Technology*, *4*(2), Article 024001. https://doi.org/10.1088/2058-9565/aaea94

Javadi-Abhari, A., Treinish, M., Krsulich, K., Wood, C. J., Lishman, J., Gacon, J., Martiel, S., Nation, P. D., Bishop, L. S., Cross, A. W., Johnson, B. R., & Gambetta, J. M. (2024). *Quantum computing with Qiskit* (arXiv:2405.08810). arXiv. https://doi.org/10.48550/arXiv.2405.08810

McClean, J. R., Boixo, S., Smelyanskiy, V. N., Babbush, R., & Neven, H. (2018). Barren plateaus in quantum neural network training landscapes. *Nature Communications*, *9*, Article 4812. https://doi.org/10.1038/s41467-018-07090-4

Minsky, M., & Papert, S. (1969). *Perceptrons: An introduction to computational geometry*. MIT Press.

Nelder, J. A., & Mead, R. (1965). A simplex method for function minimization. *The Computer Journal*, *7*(4), 308–313. https://doi.org/10.1093/comjnl/7.4.308

Nielsen, M. A., & Chuang, I. L. (2010). *Quantum computation and quantum information* (10th anniversary ed.). Cambridge University Press. https://doi.org/10.1017/CBO9780511976667

Preskill, J. (2018). Quantum computing in the NISQ era and beyond. *Quantum*, *2*, 79. https://doi.org/10.22331/q-2018-08-06-79

Quantum logic gate. (n.d.). In *Wikipedia*. Retrieved October 31, 2026, from https://en.wikipedia.org/wiki/Quantum_logic_gate

Quokka Computing. (n.d.-a). *Quokka start here* [Google Colab notebook]. Retrieved October 31, 2026, from https://colab.research.google.com/drive/17sM33S5XXjiew5Ss9U00ygOqLNSUUMOP

Quokka Computing. (n.d.-b). *Quokka docs* [Google Colab notebook]. Retrieved October 31, 2026, from https://colab.research.google.com/drive/1hRRnMnF9ivumrtVbku2dMlHtrRULpIav

Schuld, M., & Petruccione, F. (2021). *Machine learning with quantum computers* (2nd ed.). Springer. https://doi.org/10.1007/978-3-030-83098-4

Spall, J. C. (1992). Multivariate stochastic approximation using a simultaneous perturbation gradient approximation. *IEEE Transactions on Automatic Control*, *37*(3), 332–341. https://doi.org/10.1109/9.119632

University of Technology Sydney. (2024, June 25). *Quokka: A quantum hop into the future of computing*. https://www.uts.edu.au/news/2024/06/quokka-quantum-hop-future-computing

Vedral, V., Barenco, A., & Ekert, A. (1996). Quantum networks for elementary arithmetic operations. *Physical Review A*, *54*(1), 147–153. https://doi.org/10.1103/PhysRevA.54.147

Yanofsky, N. S., & Mannucci, M. A. (2008). *Quantum computing for computer scientists*. Cambridge University Press. https://doi.org/10.1017/CBO9780511813887

Appendix A – Submission Structure and How to Run

```
quantumcomputing_YB_Anish/            (GitHub repository)
├── README.md                         step-by-step run instructions
└── Assignment_2_Quantum/
    ├── Task_1_Bell_State/            notebook + results/
    ├── Task_2_Qiskit_QASM/           notebook, custom_half_adder.qasm, results/
    ├── Task_3_Quantum_TicTacToe/     completed + original notebook, results/
    ├── Task_4_Quantum_ML/            modified + original notebook, results/
    ├── Alternative_Code/             second solution for each task + outputs/
    ├── common/quokka_helper.py       shared Quokka / fallback / counts
    ├── Report/                       Assignment2.docx, Assignment2.pdf
    ├── run_project.py  run_project.bat  run_all.bat  run_project.sh
    ├── Colab_Run_All_Tasks.ipynb     whole project in Google Colab
    └── requirements.txt  MAINTENANCE.md
```

To reproduce every result: double-click `run_all.bat` (Windows), run `python run_project.py all` (any computer) or open `Colab_Run_All_Tasks.ipynb` in Google Colab and choose Runtime → Run all. Each `results/` folder then holds the JSON (raw counts, source, time), CSV and PNG evidence used in this report, and `outputs_html/` holds a web-page copy of every notebook. Repository: https://github.com/anishnz/quantumcomputing_YB_Anish. `MAINTENANCE.md` explains common errors and safe changes.

Appendix B – Key Code Excerpts

B.1 Sending a circuit to Quokka with a labelled fallback (common/quokka_helper.py, shortened)

```
def run_qasm(qasm_text, shots=1000, label="run", save_dir=None,
             allow_fallback=True):
    try:
        data = run_on_quokka(qasm_text, shots)    # POST script + count
        shot_list = list(data["result"].values())[0]   # [c0, c1] per shot
        source = SOURCE_QUOKKA
    except Exception as exc:                      # offline / error reply
        if not allow_fallback: raise
        shot_list = run_locally(qasm_text, shots) # SAME script on Aer
        source = SOURCE_LOCAL                     # never called Quokka data
    counts = shots_to_counts(shot_list)           # {'00': 508, '11': 492}
```

B.2 Completed moves in the tic-tac-toe game (Task 3, Board.make_move)

```
if self.function == 'Not':
    self.qc.x(cell)                    # [ADDED] bit flip: swap ownership
elif self.function == 'O':
    self.qc.ry(ROTATION, cell)         # [ADDED] quarter turn toward |1>
elif self.function == 'X':
    self.qc.ry(-ROTATION, cell)        # [ADDED] quarter turn toward |0>
elif (self.function == 'SWAP' and self.target >= 0
      and self.target != cell):        # [FIXED] needs a first tile
    self.qc.swap(cell, self.target)    # [ADDED] exchange two tiles
    self.target = -1                   # [FIXED] clear the first tile
```

B.3 Per-iteration monitoring added to SPSA (Task 4)

```
t0 = time.perf_counter()                          # [ADDED] start timer
grad = (func(xp) - func(xm)) / (2 * ck) * delta   # 2 evaluations
x = x - ak * grad                                 # update parameters
step_time = time.perf_counter() - t0              # [ADDED] iteration time
```

```
fx = func(x)                                        # [ADDED] output measure
history.append({"iteration": k + 1, "output": fx,
                "time_s": step_time})
```